ML & Data Science Interview Preparat

Amazon & Microsoft Applied Scientist / ML Researcher

■ Topics	■ Algorithms	■ Q&A Pairs	■ Study Weeks
10+ Areas	**40+ Covered**	**100 Detailed**	**8 Weeks**

This guide covers every topic an Applied Scientist needs to ace Amazon and Microsoft interviews: core ML theory, practical algorithms, deep learning and transformers, statistics and A/B testing, causal inference, recommender systems, system design, coding patterns, and 100 top interview questions with full answers.

How to use: Follow the chapter order — each section builds on the last. Figures make the concepts visual. Code snippets show practical implementation. The Q&A; section at the end tests everything.

© Study Notes 2025

Table of Contents

Your complete ML study roadmap

1. Machine Learning Foundations ...

1.1 Bias-Variance Tradeoff
1.2 Regularisation (L1, L2)
1.3 Model Evaluation & Metrics
1.4 Optimisation & Gradient Descent
1.5 Feature Engineering

2. Core Algorithms ...

2.1 Linear & Logistic Regression
2.2 Decision Trees, Random Forests, Boosting
2.3 Support Vector Machines
2.4 Naive Bayes & KNN
2.5 Model Comparison Guide

3. Unsupervised Learning ...

3.1 K-Means, DBSCAN, GMM
3.2 PCA & Dimensionality Reduction
3.3 t-SNE & UMAP
3.4 Recommender Systems

4. Deep Learning ...

4.1 Neural Network Fundamentals
4.2 CNNs & Computer Vision
4.3 RNNs, LSTMs & Sequence Models
4.4 Transformers & Attention
4.5 LLMs, BERT, GPT, Fine-tuning

5. Natural Language Processing ...

5.1 Text Preprocessing & Tokenisation
5.2 Word Embeddings
5.3 Sequence-to-Sequence Models
5.4 RAG & Prompt Engineering

6. Statistics & Probability ...

6.1 Probability Foundations
6.2 Hypothesis Testing
6.3 Bayesian Statistics
6.4 Distributions You Must Know

© Study Notes 2025

7. A/B Testing & Causal Inference ...

7.1 Experiment Design
7.2 Statistical Power & Sample Size
7.3 CUPED Variance Reduction
7.4 Causal Inference Methods

8. ML System Design ...

8.1 Design Framework
8.2 Recommender System
8.3 Fraud Detection
8.4 Search Ranking
8.5 MLOps & Production

9. Coding Patterns ...

9.1 Python / NumPy / Pandas
9.2 PyTorch Essentials
9.3 SQL for Data Science

10. Top 100 Interview Q&A; ...

Grouped by topic with detailed answers

© Study Notes 2025

Chapter 1 — ML Foundations

Bias-variance, regularisation, evaluation, optimisation

1.1 Bias-Variance Tradeoff

ML HOT

Every model makes errors. The total expected error splits into three parts:

```
Expected Error = Bias² + Variance + Irreducible Noise

Bias     = E[■] − y          (how wrong on average — systematic error)
Variance = E[(■ − E[■])²]  (how much predictions change with training data)
```

Analogy: Imagine throwing darts. Bias = always landing left of the bullseye. Variance = landing all over the place. You want both low.

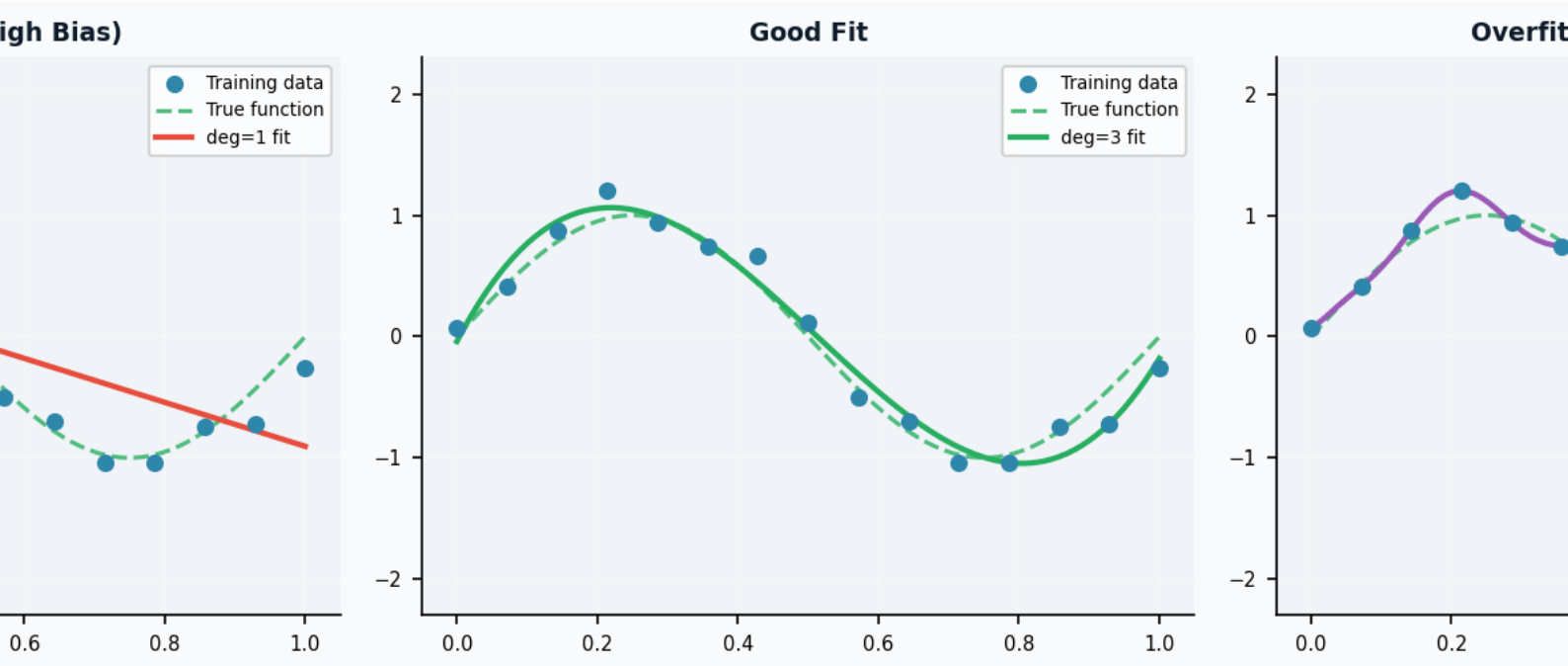

Figure 1.1: Degree-1 polynomial underfits (high bias), degree-15 overfits (high variance), degree-3 is just right.

Problem	Train Error	Val Error	Fix
High Bias (underfitting)	High	High	More features, complex model, less regularisation
High Variance (overfitting)	Low	High	More data, regularise, simpler model, dropout
Good fit	Low	Low ≈ Train	—

■ TIP

Diagnostic: Plot learning curves. If train and val error are both high and converge — bias problem. If there is a big gap between train (low) and val (high) — variance problem.

© Study Notes 2025

■ EXAMPLE
At Amazon, a product ranking model with low AUC on held-out data but perfect train AUC has overfit the training distribution. Adding L2 regularisation or more training data fixes it.

1.2 Regularisation — L1, L2, Elastic Net

ML HOT

Regularisation adds a penalty to the loss to prevent overfitting — it forces the model to stay "simple".

```
L2 Ridge: Loss = MSE + λ · Σw■²       → all weights shrink, none go to zero
L1 Lasso: Loss = MSE + λ · Σ|w■|     → some weights become exactly 0
Elastic:  Loss = MSE + λ■·Σ|w■| + λ■·Σw■²  → combines both
```

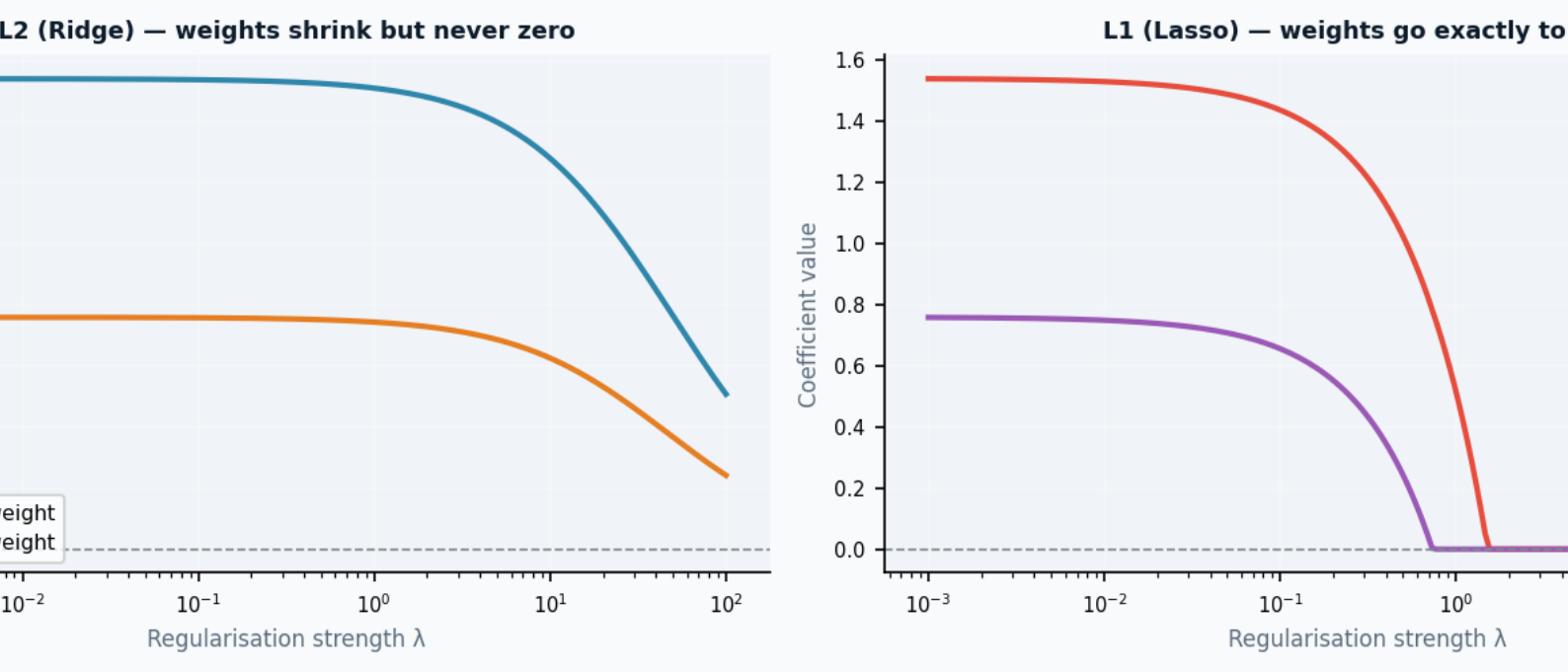

Figure 1.2: L2 shrinks all weights smoothly. L1 creates sparse models (some weights hit exactly zero) — acting as automatic feature selection.

■ KEY IDEA
Why does L1 give sparsity? The L1 ball is diamond-shaped with sharp corners on the axes. The loss contour is most likely to first touch one of these corners, forcing a weight to zero. L2 has a smooth circular ball — no corners — so weights stay small but non-zero.

■ EXAMPLE
Real-world: You are building a click-through rate predictor with 50,000 user features. Lasso will automatically zero out 48,000 irrelevant features, keeping only the most predictive ones. Ridge keeps all features but makes most very small.

© Study Notes 2025

```python
from sklearn.linear_model import Ridge, Lasso, ElasticNet

# Ridge: all features kept, coefficients shrunk
ridge = Ridge(alpha=1.0)  # lambda = alpha
ridge.fit(X_train, y_train)
print("Ridge: zero coefs:", (ridge.coef_ == 0).sum())  # 0

# Lasso: automatic feature selection
lasso = Lasso(alpha=0.01)
lasso.fit(X_train, y_train)
print("Lasso: zero coefs:", (lasso.coef_ == 0).sum())  # many!

# ElasticNet: best of both
en = ElasticNet(alpha=0.01, l1_ratio=0.5)  # 50% L1, 50% L2
en.fit(X_train, y_train)
```

1.3 Model Evaluation & Metrics

ML HOT

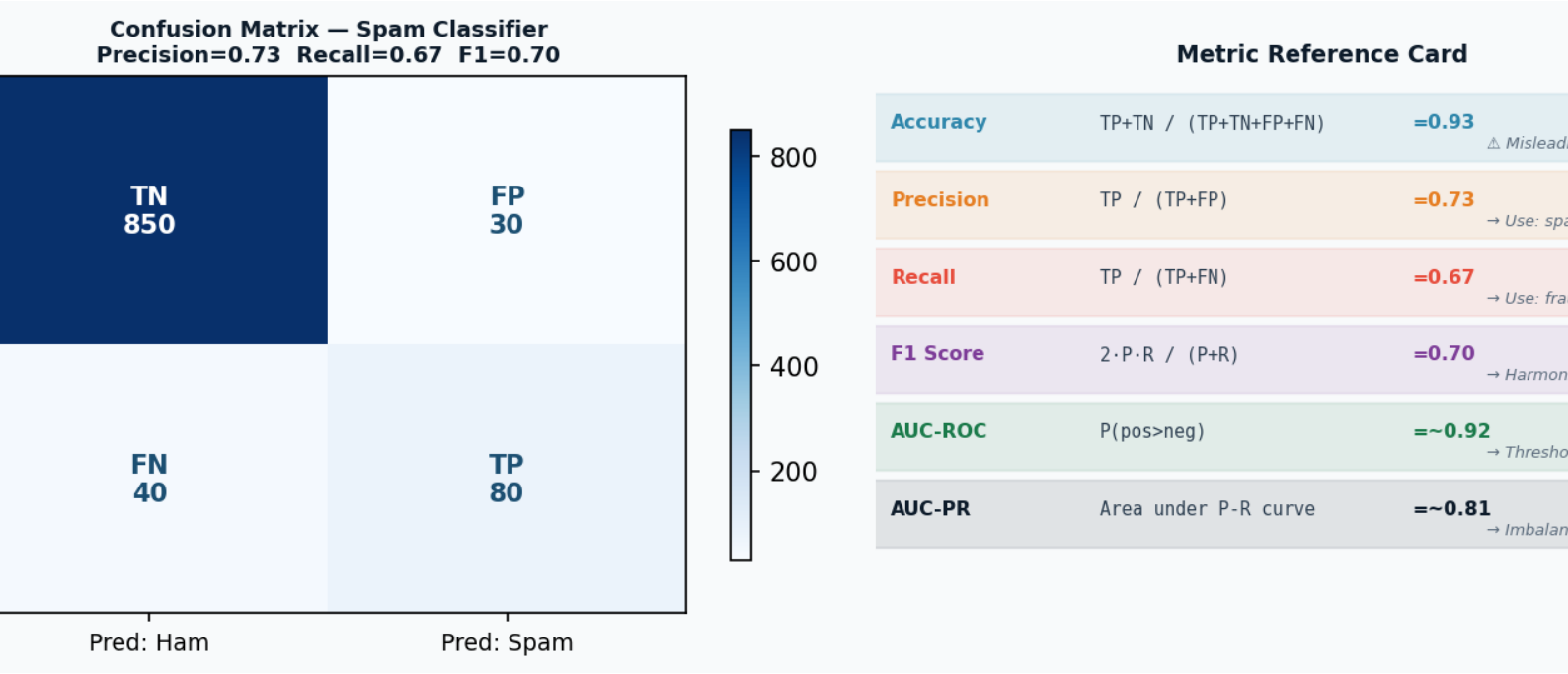

Figure 1.3: Confusion matrix for a spam classifier (1000 emails). Precision = how many flagged spams are actually spam. Recall = how many real spams did we catch.

Choosing the wrong metric is a career-ending mistake in interviews. Match the metric to the business cost:

Metric	Formula	Use When	Example
Accuracy	(TP+TN)/N	Balanced classes	MNIST digit classification
Precision	TP/(TP+FP)	FP is costly	Spam filter (blocking real email is bad)
Recall	TP/(TP+FN)	FN is costly	Cancer screening (missing cancer is bad)
F1 Score	2·P·R/(P+R)	Need balance	Named entity recognition
AUC-ROC	P(pos>neg)	Balanced data, threshold-free	Fraud scoring (ranked output)

© Study Notes 2025

Metric	Formula	Use When	Example
AUC-PR	Area P-R curve	Imbalanced (< 10% positive)	Ad click prediction (0.1% CTR)
NDCG@k	Position-weighted gain	Ranking matters	Search results, recommendations

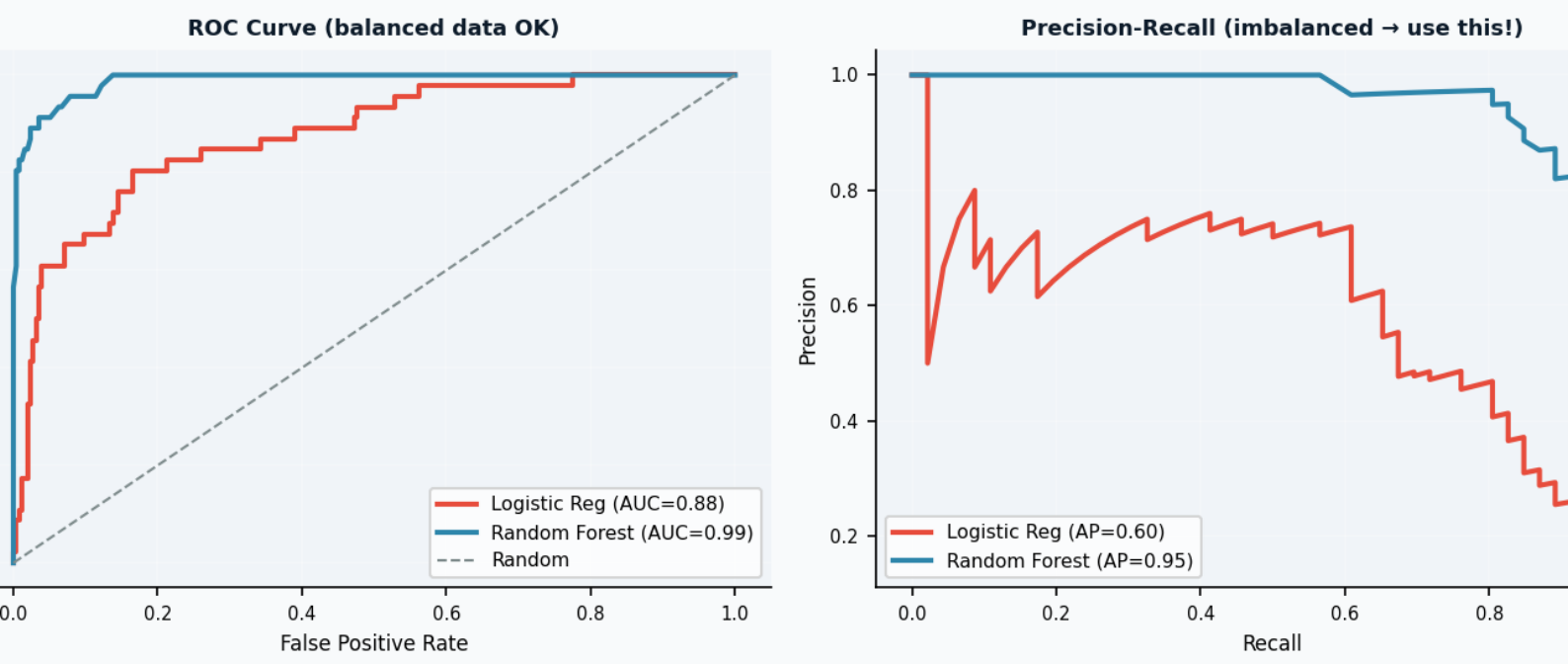

Figure 1.4: With imbalanced data (85% negative class), AUC-ROC looks decent but AUC-PR reveals the true story — use AUC-PR for fraud, disease, and ad click prediction.

Cross-Validation

Never evaluate on training data. Use these strategies:

Method	When to Use	Notes
k-fold (k=5 or 10)	Default choice	Balance of bias and compute
Stratified k-fold	Imbalanced classes	Keeps class ratio per fold
Time series split	Sequential/temporal data	NEVER shuffle time series!
Leave-one-out	Very small datasets (n<50)	Expensive, low bias

■ NOTE

Never use regular k-fold for time series! If your model predicts tomorrow using tomorrow's data from a shuffled fold, you have data leakage and meaningless results.

© Study Notes 2025

```python
from sklearn.model_selection import StratifiedKFold, TimeSeriesSplit
from sklearn.metrics import roc_auc_score
import numpy as np

# Stratified CV for imbalanced classification
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = []
for train_idx, val_idx in cv.split(X, y):
    X_tr, X_val = X[train_idx], X[val_idx]
    y_tr, y_val = y[train_idx], y[val_idx]
    model.fit(X_tr, y_tr)
    scores.append(roc_auc_score(y_val, model.predict_proba(X_val)[:,1]))
print(f"AUC: {np.mean(scores):.3f} ± {np.std(scores):.3f}")

# Time series split — always train on past, predict future
tscv = TimeSeriesSplit(n_splits=5)
for train_idx, val_idx in tscv.split(X):
    # train_idx always < val_idx (no future leakage)
    pass
```

1.4 Optimisation — Gradient Descent & Variants

ML HOT

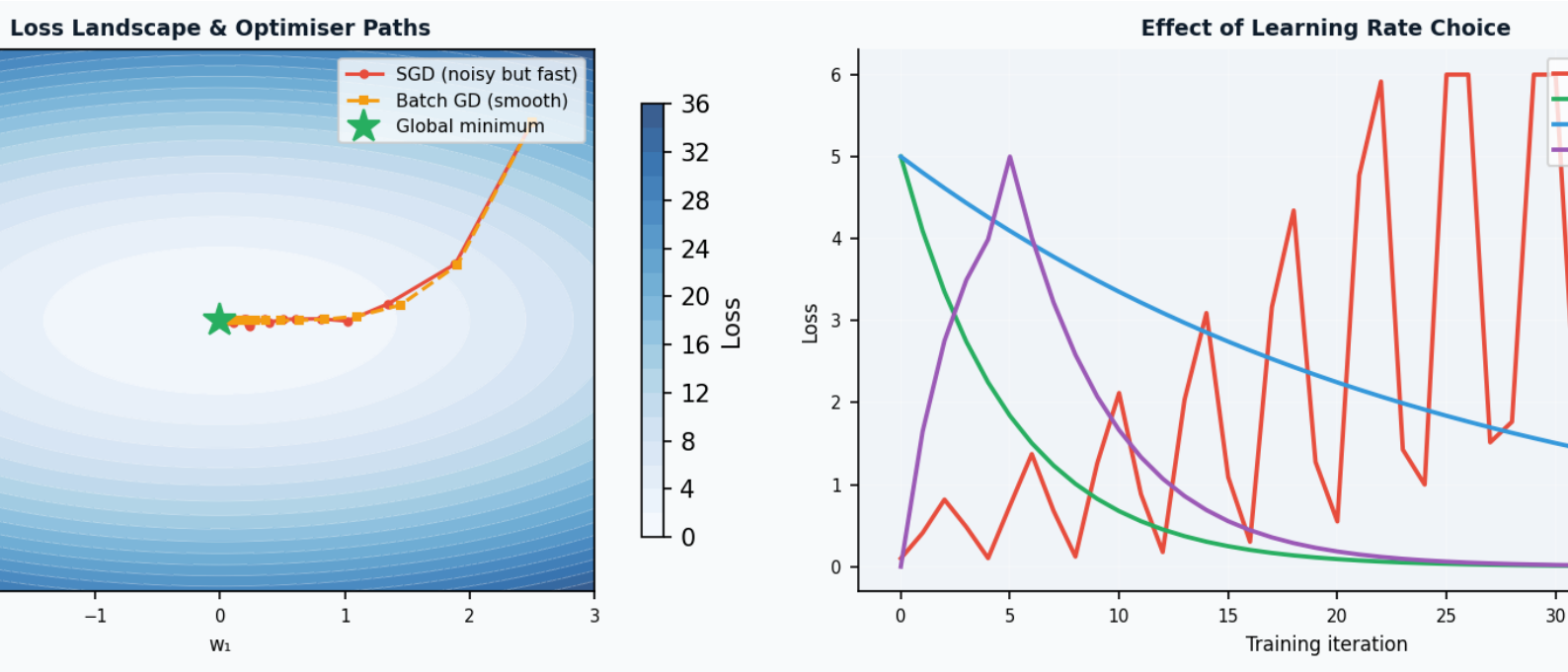

Figure 1.5: Left — SGD takes a noisy zigzag path; Batch GD is smooth. Right — too-high LR diverges, too-low is slow. Warmup (start small, ramp up) is critical for transformer training.

Gradient descent update: $w \leftarrow w - \eta \cdot \nabla L(w)$

Adam (best general-purpose optimizer):

$m \leftarrow \beta_1 m + (1-\beta_1)\nabla L$ (momentum — smooth gradients)

$v \leftarrow \beta_2 v + (1-\beta_2)(\nabla L)^2$ (adaptive lr — scale by gradient magnitude)

$m_\square = m/(1-\beta_1^\square)$ $v_\square = v/(1-\beta_2^\square)$ (bias correction)

$w \leftarrow w - \eta \cdot m_\square / (\sqrt{v_\square} + \varepsilon)$

© Study Notes 2025

Optimizer	Adaptive LR	Momentum	Best For	Avoid When
SGD + momentum	No	Yes	CNNs (with tuning, best generalisation)	Quick experiments
Adam	Yes	Yes	NLP, general deep learning	Small tabular data
AdamW	Yes	Yes (decoupled WD)	Transformers (GPT, BERT)	—
RMSProp	Yes	No	RNNs, non-stationary problems	—
Adagrad	Yes (decays)	No	Sparse features, NLP embedding tuning	Long training runs

■ TIP

Why AdamW for transformers? Standard Adam conflates weight decay with gradient scaling. AdamW separates them: $w \leftarrow (1-\lambda\eta)w - \eta \cdot m_{\blacksquare}/(\sqrt{v_{\blacksquare}}+\varepsilon)$. This makes regularisation predictable regardless of gradient scale.

1.5 Feature Engineering & Handling Data

ML

The most impactful thing you can do is build better features. Models are limited by the information in their inputs.

Encoding Categorical Variables

Method	When to Use	Watch Out
One-hot encoding	Low cardinality (< 20 categories)	Creates sparse high-dim data
Label encoding	Ordinal features (Small < Med < Large)	Never for nominal categories
Target encoding	High cardinality (city, product ID)	Data leakage — fit inside CV folds only!
Embeddings (DL)	Very high cardinality (user IDs, words)	Needs sufficient data to learn

Missing Values

Type	Meaning	Strategy
MCAR — Missing Completely At Random	No pattern to missingness	Mean/median/mode imputation
MAR — Missing At Random	Missingness depends on other features	Model-based imputation (KNN, IterativeImputer)
MNAR — Missing Not At Random	Missingness tells you something	Add is_missing flag feature + impute

© Study Notes 2025

```python
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer, KNNImputer

# ALWAYS fit imputer on training data only, transform both
imputer = KNNImputer(n_neighbors=5)
X_train_imp = imputer.fit_transform(X_train)
X_test_imp  = imputer.transform(X_test)  # no fit here!

# Target encoding — CORRECT (inside CV fold to prevent leakage)
def target_encode(X_tr, y_tr, X_val, col):
    mapping = X_tr.groupby(col)["target"].mean()
    global_mean = y_tr.mean()
    X_tr[col + "_enc"] = X_tr[col].map(mapping).fillna(global_mean)
    X_val[col + "_enc"] = X_val[col].map(mapping).fillna(global_mean)
    return X_tr, X_val
```


© Study Notes 2025

Chapter 2 — Core Algorithms

Every algorithm you need, with comparisons

2.1 Linear & Logistic Regression

ML

```
Linear regression:  ■ = Xw + b
Loss (MSE):         L = (1/n) Σ(y■ − ■■)²
Closed-form:        w* = (X■X)■¹X■y  (only for small feature count)

Logistic regression: P(y=1|x) = σ(w■x + b) where σ(z) = 1/(1+e■■)
Loss (BCE):          L = −[y·log(■) + (1−y)·log(1−■)]
Gradient:            ∂L/∂w = (1/n) X■(■ − y)  ← same elegant form as linear!
```

■ KEY IDEA

Logistic regression is a linear classifier — it can only separate data with a linear boundary. But it gives calibrated probabilities, is fast, and is the right first baseline for any classification task.

python

```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    C=1.0,          # C = 1/lambda — larger C = less regularisation
    penalty="l2",   # L2 by default
    solver="lbfgs", # or "saga" for L1
    max_iter=1000
)
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1]  # P(class=1)
```

2.2 Decision Trees, Random Forests & Gradient Boosting

ML HOT

© Study Notes 2025

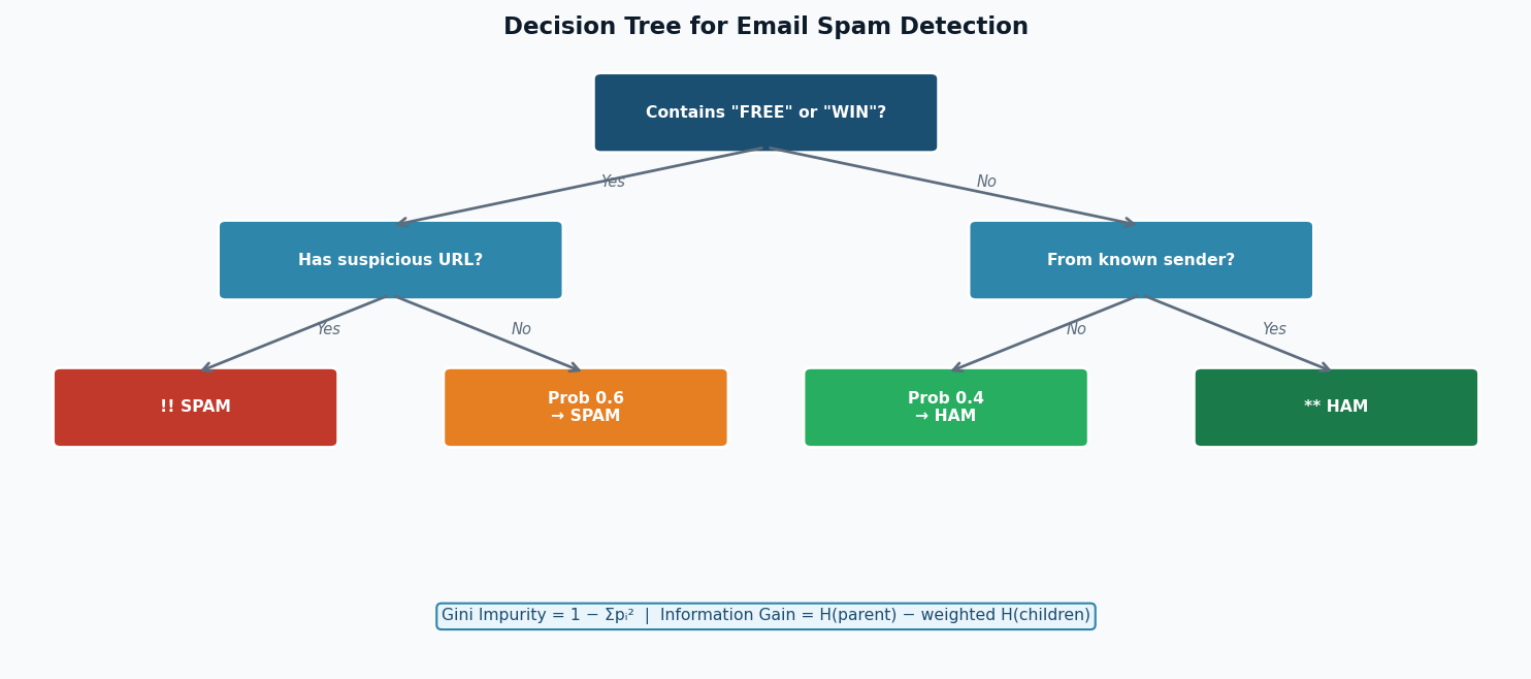

Figure 2.1: Decision tree for spam detection. Each node splits on the feature that best separates the classes, measured by Gini impurity or information gain.

```
Gini Impurity:     G(S) = 1 − Σ p■²
Information Gain:  IG(S, A) = H(S) − Σ (|S■|/|S|)·H(S■)
Variance Reduction (regression): Var(parent) − weighted average Var(children)
```

Random Forest — Why Averaging Works

Random Forest trains many trees on different random subsets of data (bagging) and features. The key insight: errors are random per tree, so averaging them out reduces variance without touching bias.

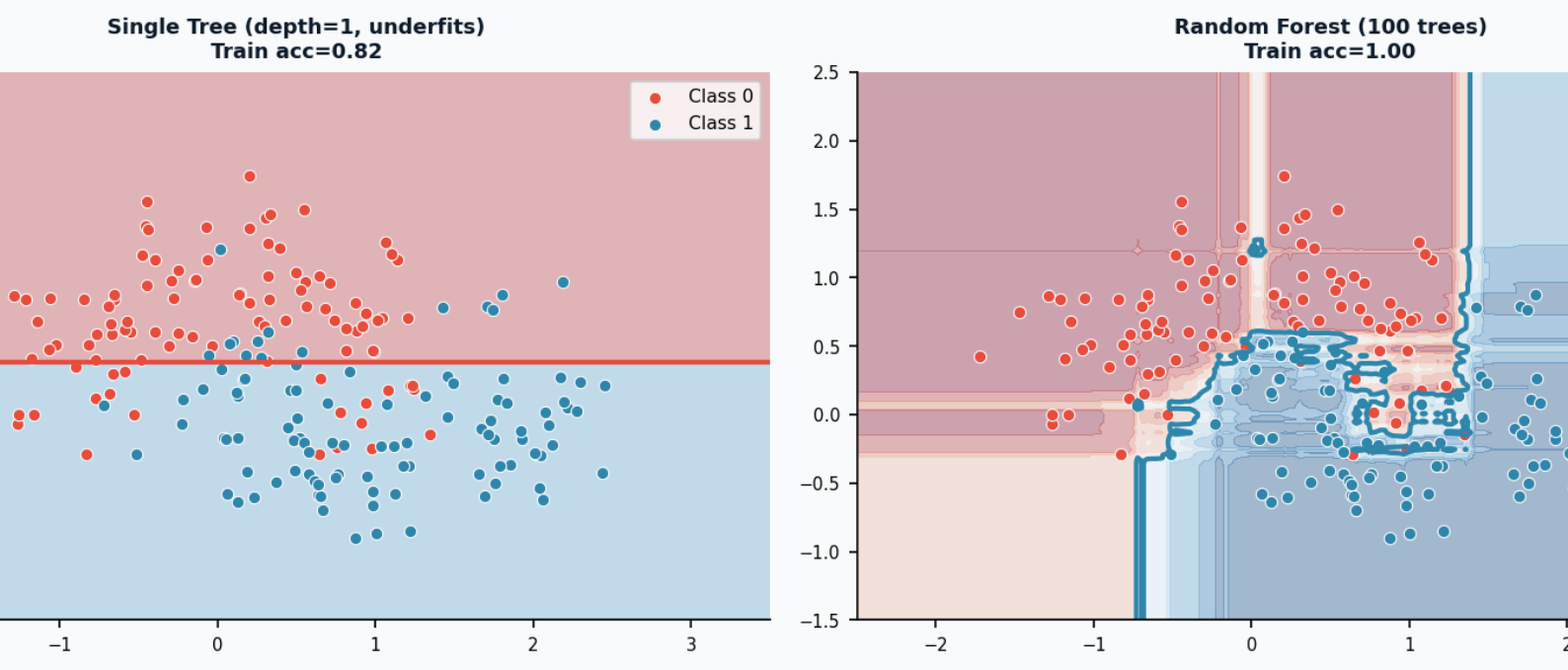

Figure 2.2: A single shallow tree cannot capture the curved boundary (high bias). 100 trees averaged together get the shape right — bagging reduces variance.

© Study Notes 2025

Gradient Boosting — Fixing Mistakes Sequentially

```
F■(x) = mean(y)   (start with a constant)
F■(x) = F■■■(x) + η · h■(x)  where h■ fits residuals r■ = y■ – F■■■(x■)

General form: r■ = –∂L/∂F(x■)|_{F=F■■■}  (negative gradient)
```

■ EXAMPLE

Imagine predicting house prices. First tree guesses $300k for all houses. Residuals are computed — some houses need +$100k, others –$50k. Next tree learns to predict these residuals. Repeat 500 times. Each tree corrects the mistakes of all previous trees.

Method	Trees Grow	Speed	Key Strengths	Watch Out
Random Forest	In parallel	Fast	Robust, low variance, easy to tune	Less accurate than boosting on most task
XGBoost	Sequential, level-wise	Fast	L1+L2 regularised, handles missing values	Many hyperparameters
LightGBM	Sequential, leaf-wise	Fastest (10-100x XGB)	Huge datasets, categorical features natively	Can overfit small datasets
CatBoost	Sequential, symmetric	Fast	Best for categoricals, ordered boosting	Slower than LightGBM

■ AMAZON/MSFT

At Amazon, LightGBM/XGBoost is the go-to for tabular data (ads CTR prediction, demand forecasting, fraud scoring). Interviewers expect you to know: which hyperparameters matter (n_estimators, learning_rate, max_depth, subsample), how to avoid overfitting, and what SHAP values are.

python

```python
import lightgbm as lgb

model = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.05,        # lower = more trees needed but better
    max_depth=6,               # control overfitting
    num_leaves=31,             # should be < 2^max_depth
    subsample=0.8,             # row subsampling
    colsample_bytree=0.8,      # feature subsampling
    reg_alpha=0.1, reg_lambda=1.0,  # L1, L2
    class_weight="balanced"   # for imbalanced data
)
model.fit(X_tr, y_tr,
          eval_set=[(X_val, y_val)],
          callbacks=[lgb.early_stopping(50), lgb.log_evaluation(100)])

# SHAP values — explain every prediction
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

2.3 Support Vector Machines (SVM)

ML

© Study Notes 2025

```
Goal: find hyperplane w■x + b = 0 that maximises margin = 2/||w||
Primal:  min ½||w||²  subject to  y■(w■x■ + b) ≥ 1
Soft margin:  min ½||w||² + C·Σξ■  (allow some misclassifications)
Kernel trick: replace x■z with K(x,z) = ϕ(x)■ϕ(z)  (no explicit ϕ needed!)
```

Kernel	Formula	Use When
Linear	x■z	Text classification, high-dim data
RBF/Gaussian	exp(–γ||x–z||²)	Default choice, non-linear data
Polynomial	(γx■z + r)■	Image features, bounded input

■ **NOTE**
SVM scales as O(n² to n³). For more than 100k samples, use SGDClassifier with hinge loss instead — same decision boundary, much faster.

2.5 Model Comparison — When to Use What

ML HOT

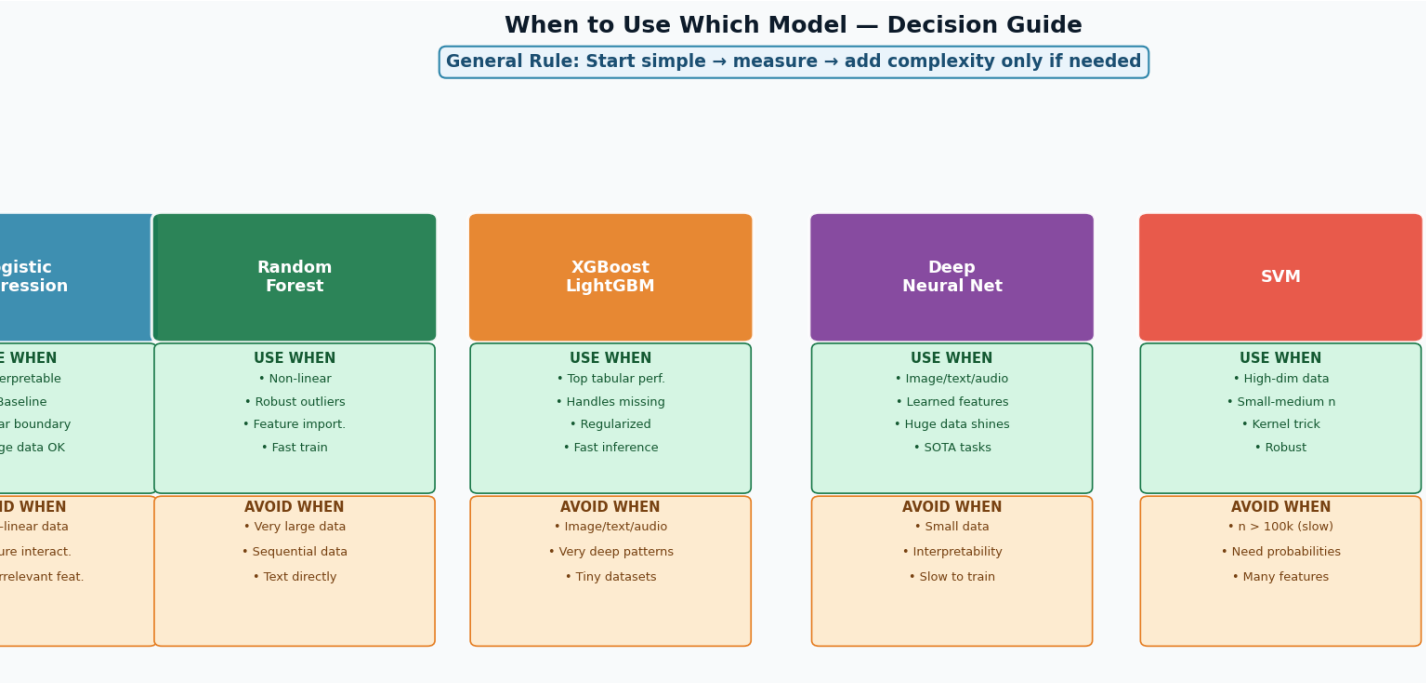

Figure 2.3: Decision guide for picking the right model. Always start with a simple baseline (logistic regression or linear SVM) before adding complexity.

© Study Notes 2025

Chapter 3 — Unsupervised Learning

Clustering, dimensionality reduction, recommenders

3.1 Clustering

ML

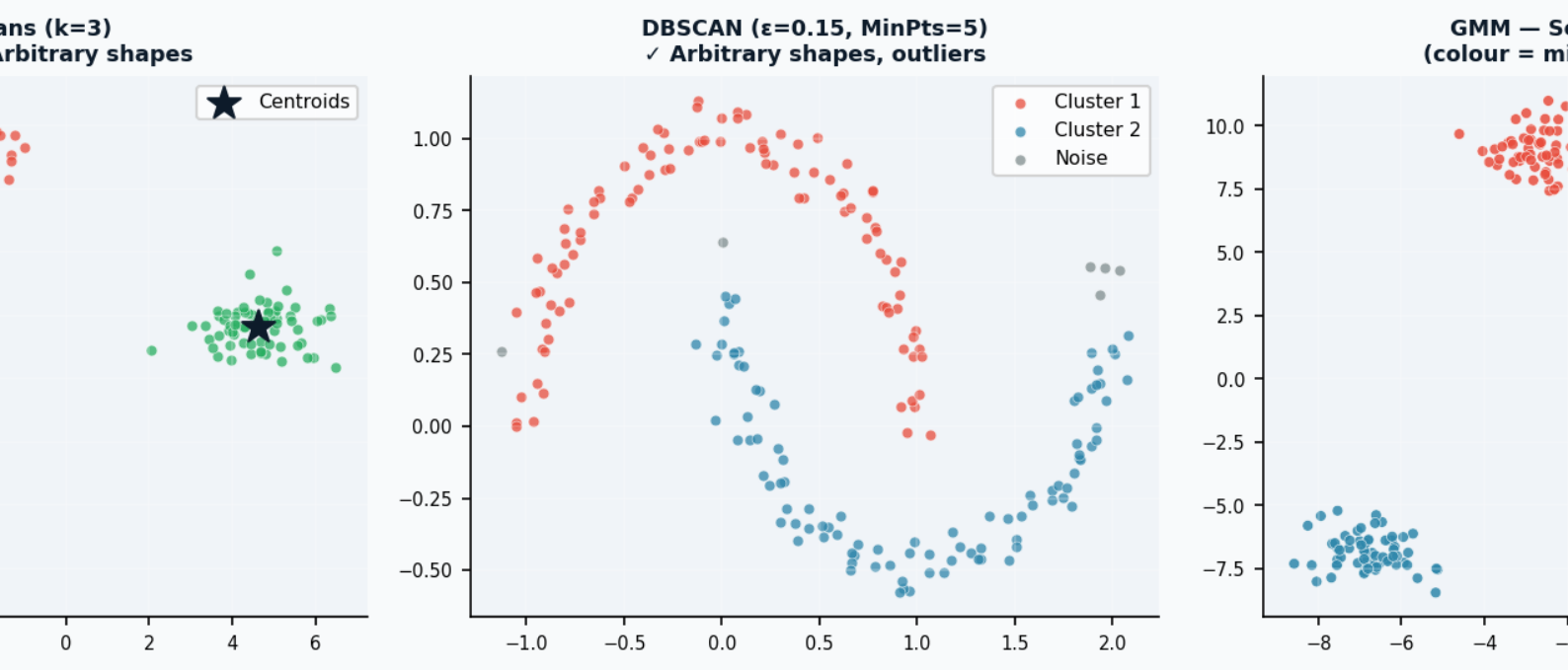

Figure 3.1: K-Means finds convex clusters. DBSCAN finds arbitrary shapes and labels outliers as noise. GMM gives soft probabilistic assignments — each point belongs to each cluster with some probability.

```
K-Means objective:  min Σ■ Σ■∈C■ ||x − μ■||²
E-step: assign each point to nearest centroid
M-step: recompute centroids as mean of assigned points

GMM: p(x) = Σ■ π■ · N(x; μ■, Σ■)  where Σπ■ = 1
```

Algorithm	Shape	Outliers	Need k?	Soft assignments?
K-Means	Convex (circular)	Sensitive	Yes	No
DBSCAN	Any	Labels as noise	No (use ε, MinPts)	No
GMM	Elliptical	Moderate	Yes	Yes (probabilistic)
Hierarchical	Any	Moderate	Post-hoc	No

Evaluating Clusters (No Labels)

© Study Notes 2025

```
Silhouette Score: s(i) = (b−a) / max(a,b)  range [−1, 1], higher=better
  a = avg distance to same cluster points
  b = avg distance to nearest other cluster
Davies-Bouldin: lower is better (within/between cluster ratio)
```

```python
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# ALWAYS scale before clustering
X_scaled = StandardScaler().fit_transform(X)

# Find optimal k with elbow method
inertias = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
# Plot inertias — look for the "elbow" point

# DBSCAN — no k needed
db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X_scaled)
print(f"Clusters: {len(set(labels)) - (1 if -1 in labels else 0)}")
print(f"Noise: {(labels == -1).sum()}")
print(f"Silhouette: {silhouette_score(X_scaled, labels):.3f}")
```

3.2 PCA & Dimensionality Reduction

ML HOT

PCA finds the directions of maximum variance. It is an indispensable tool for visualisation, noise reduction, and speeding up downstream models.

```
Steps:
1. Centre data: X■ = X − mean(X)
2. Covariance matrix: Σ = X■■X■ / (n−1)
3. Eigenvectors of Σ = principal components
4. Project: Z = X■ · V_k  (keep top-k eigenvectors)

Equivalent via SVD: X = UΣV■  →  principal components = columns of V
```


© Study Notes 2025

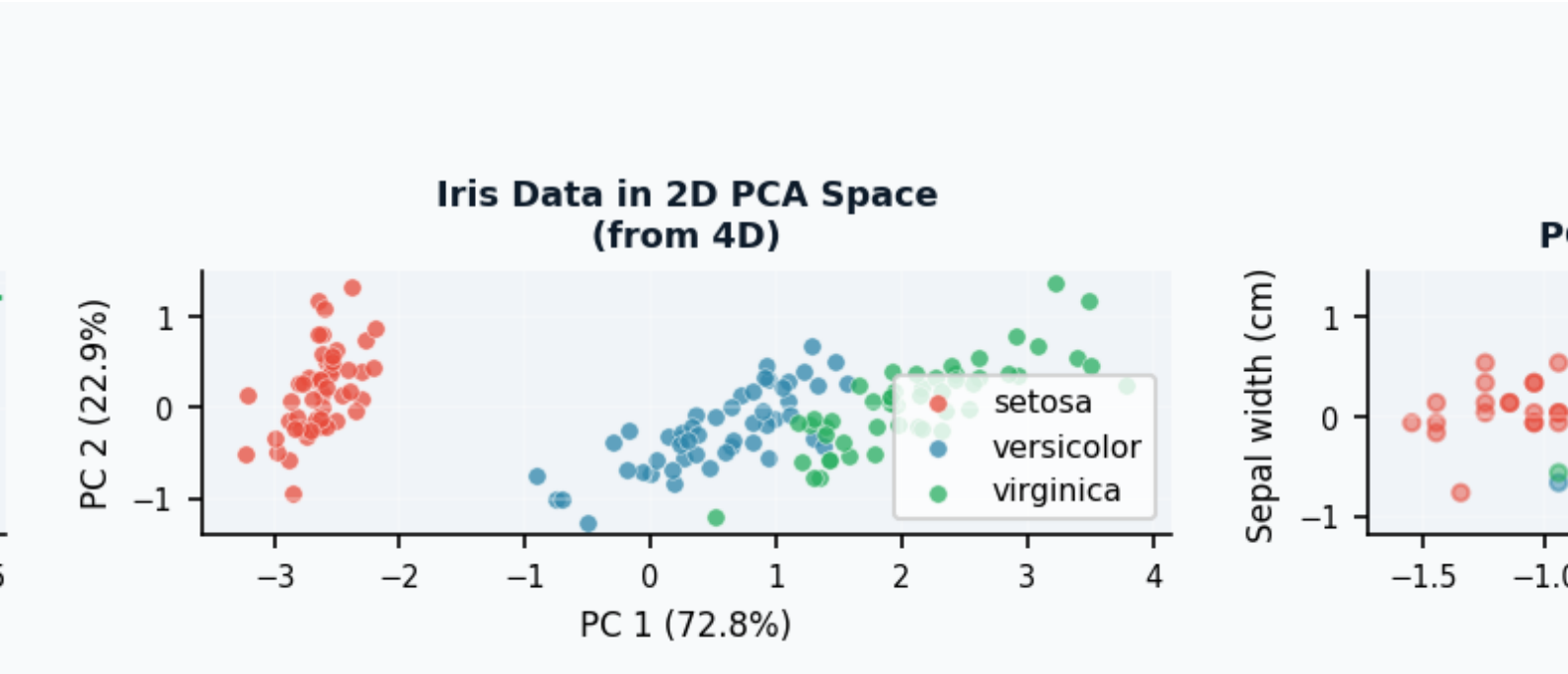

Figure 3.2: Left — scree plot shows 2 PCs explain 97.8% variance in 4D iris data. Middle — 3 species clearly separate in 2D PCA space. Right — PCA directions point toward maximum spread in the original data.

Method	Type	Preserves	Speed	Use For
PCA	Linear	Global variance structure	$O(nd^2)$	Preprocessing, noise removal, visualisation
t-SNE	Non-linear	Local structure (clusters)	$O(n^2)$ — slow	Visualisation of clusters only
UMAP	Non-linear	Local + some global	O(n log n) — fast	Large datasets, visualisation + preprocessing
Autoencoder	Non-linear	Reconstruction quality	Depends on depth	Non-linear compression, generative

■ NOTE
t-SNE and UMAP cluster SHAPES and POSITIONS are not meaningful. Two clusters appearing close does not mean they are similar. Only use for visual exploration, never for distance-based inference.

3.4 Recommender Systems

ML HOT

Recommender systems are core to Amazon (product recs), Microsoft (content, ads). Know these approaches deeply.

Approach	How it Works	Cold Start?	Best For
Collaborative Filtering	Users who bought X also bought Y — learns from interactions/ratings	Fails for new users/items	Large user base, dense interactions
Content-Based	Recommend items similar to what user liked, based on item features	Handles new items	Rich item metadata, news, articles
Matrix Factorisation	Decompose rating matrix R ≈ PQ■ (user embeddings)	Fails for new	Netflix-style ratings
Two-Tower DNN	User tower + item tower, dot-product similarity	Add features for cold start	Web-scale retrieval (Google, Amazon)
Hybrid	Combine CF + content-based + popularity	Handles cold start	Production systems

© Study Notes 2025

```
Matrix Factorisation:
R ≈ P · Q■   (P: n_users×k, Q: n_items×k, k=latent factors)
Loss: min Σ(r■■ − p■■q■)² + λ(||P||² + ||Q||²)
Update (SGD): p■ ← p■ + η(e■■q■ − λp■)  where e■■ = r■■ − p■■q■
```

■ TIP

Two-tower architecture: encode user history into user embedding, encode item features into item embedding, score = cosine similarity. For serving, use approximate nearest neighbour (FAISS) for real-time lookup. Trained with in-batch negatives or sampled negatives.

© Study Notes 2025

Chapter 4 — Deep Learning

Neural nets, CNNs, RNNs, Transformers, LLMs

4.1 Neural Network Fundamentals

DL HOT

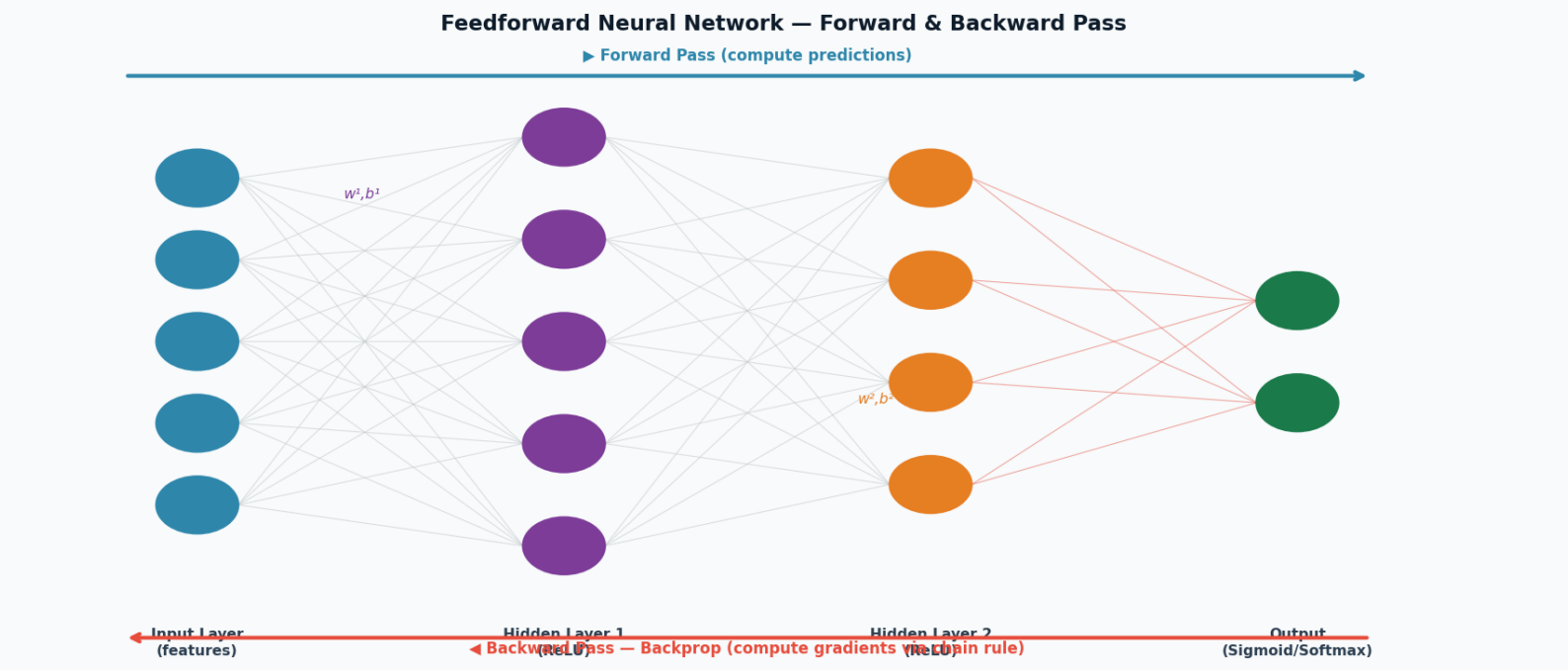

Figure 4.1: Forward pass computes predictions left-to-right. Backward pass (backpropagation) computes gradients via chain rule right-to-left to update weights.

```
Forward pass (one layer):  z = Wx + b,   a = σ(z)
Backward (chain rule):    ∂L/∂W = ∂L/∂a · ∂a/∂z · ∂z/∂W = δ · x■
  where δ = ∂L/∂z  ("error signal" flowing backward)
  ∂L/∂x■ = W■■■■ · δ■■■  (propagate error to previous layer)
```

Activation Functions

Function	Formula	Range	Problem	Use In
Sigmoid	1/(1+e■■)	(0,1)	Vanishing grad (saturates)	Output for binary classification
Tanh	(e■–e■■)/(e■+e■■)	(–1,1)	Vanishing grad (less bad)	RNNs, zero-centred
ReLU	max(0,x)	[0,∞)	Dying ReLU (dead neuron for x<0)	CNNs, hidden layers
GELU	x·Φ(x)	(–∞,∞)	Slower to compute	BERT, GPT, transformers
SiLU/Swish	x·σ(x)	(–∞,∞)	Non-monotone	LLaMA, modern models

© Study Notes 2025

■ TIP

Why GELU in transformers? GELU is smooth (differentiable everywhere), non-monotone (can suppress some activations), and empirically outperforms ReLU in attention-based models. Think of it as a probabilistic version of ReLU.

Batch Normalisation vs Layer Normalisation

```
Batch Norm:  μ_B = mean(x■ over batch), σ²_B = var(x■)
             x■■ = (x■ − μ_B)/√(σ²_B + ε),  y■ = γx■■ + β
Layer Norm:  normalise over features (not batch) — each token independently
```

Method	Normalise Over	Use In	Why
Batch Norm	Batch dimension	CNNs, MLPs	Stable training, acts as regulariser
Layer Norm	Feature dimension	Transformers	Sequence length varies, works with batch size 1
Group Norm	Channel groups	Small-batch CNNs	Batch size 1-4 in detection/segmentation

4.2 CNNs & Computer Vision

DL

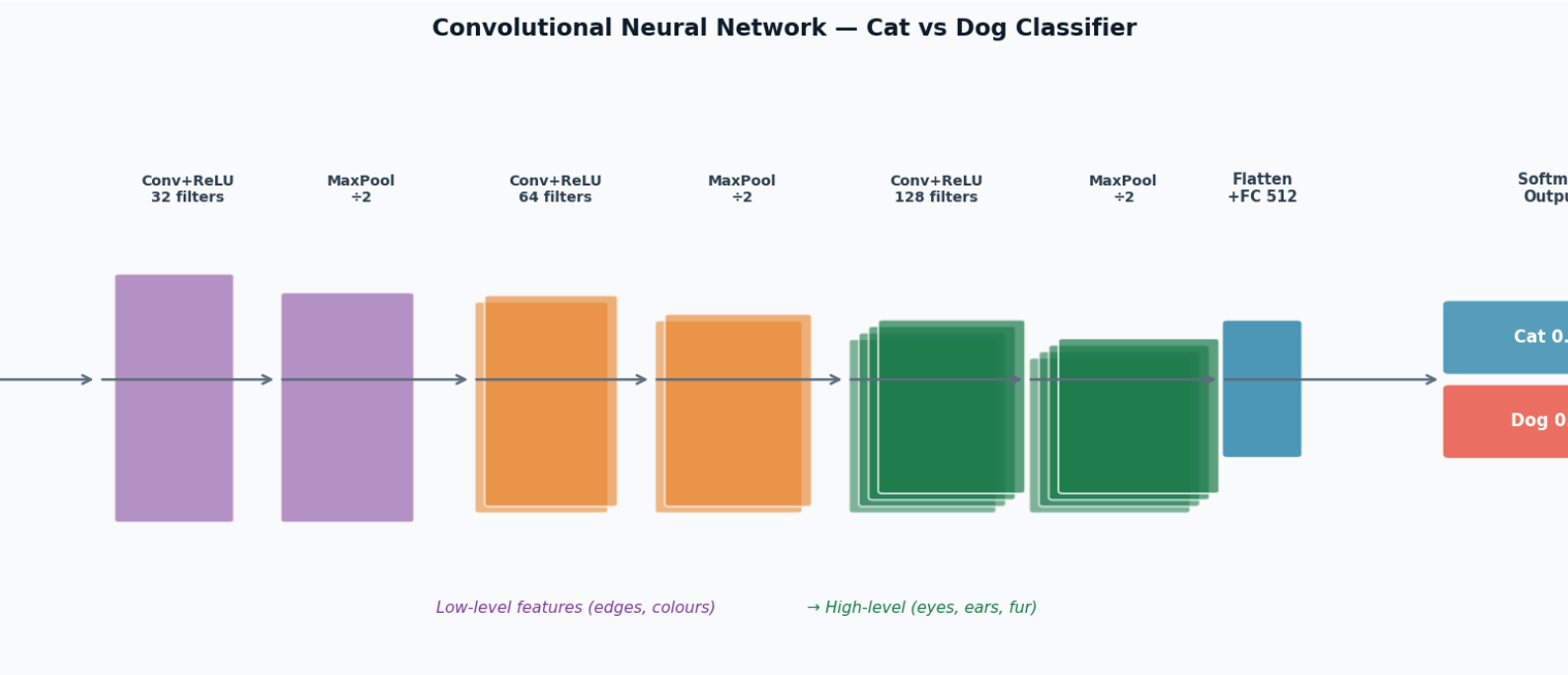

Figure 4.2: CNN for cat vs dog classification. Early layers learn edges and colours. Later layers combine these into semantic features like eyes and fur. Pooling reduces spatial size, increasing the receptive field.

© Study Notes 2025

```
Output spatial size:  H_out = ■(H_in − F + 2P) / S■ + 1
  H_in=input height, F=filter size, P=padding, S=stride

Params in conv layer:  F·F·C_in·C_out + C_out  (bias)

ResNet skip connection:  H(x) = F(x) + x
  Gradient:  ∂L/∂x = ∂L/∂H · (∂F/∂x + 1)  ← +1 prevents vanishing!
```

■ KEY IDEA

ResNet solved the degradation problem: adding more layers made things worse before ResNet. The skip connection creates a gradient highway — the +1 in the derivative ensures gradients flow even if F(x)→0. ResNet-152 with 152 layers was the first to beat human performance on ImageNet.

4.3 RNNs, LSTMs & Sequence Models

DL

```
RNN:   h■ = tanh(W■h■■■ + W■x■ + b)   — hidden state carries memory
LSTM gate equations:
  f■ = σ(Wf·[h■■■, x■] + bf)  ← forget gate: what to erase from memory
  i■ = σ(Wi·[h■■■, x■] + bi)  ← input gate: what new info to store
  c■ = f■■c■■■ + i■■tanh(Wc·[h■■■, x■] + bc)  ← cell state update
  o■ = σ(Wo·[h■■■, x■] + bo)  ← output gate
  h■ = o■ ■ tanh(c■)
```

■ TIP

Why LSTM over vanilla RNN? RNN gradients through time multiply the same weight matrix many times — they vanish or explode. LSTM cell state c■ is a "highway" where additions (not multiplications) are the dominant operation, preventing gradient vanishing. Widely replaced by transformers, but still used in streaming/edge settings.

4.4 Transformers & Self-Attention

DL HOT

© Study Notes 2025

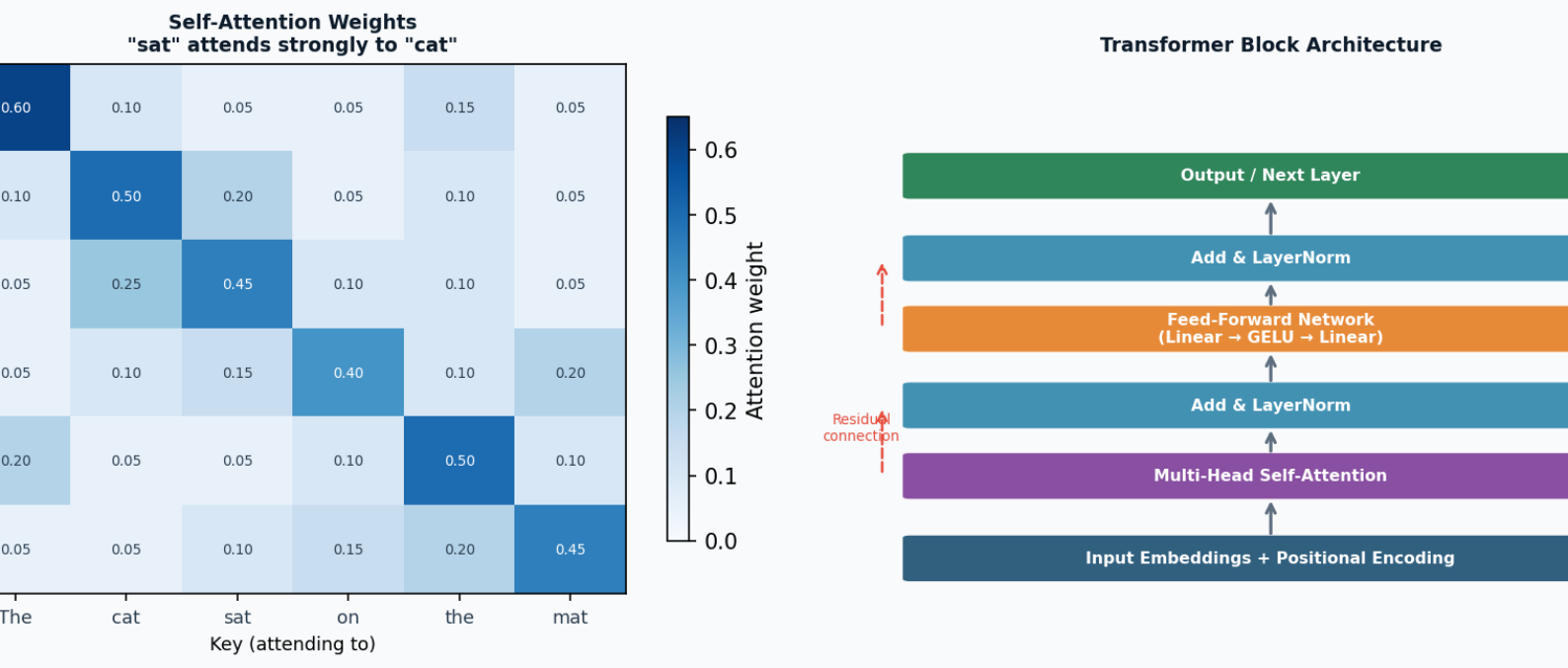

Figure 4.3: Left — attention weight matrix for "The cat sat on the mat". "sat" pays most attention to "cat" (subject). Right — transformer block: attention + residual + FFN + residual. Stacked N times.

```
Scaled dot-product attention:
  Attention(Q,K,V) = softmax(QK■ / √d_k) · V
  Q=XW_Q, K=XW_K, V=XW_V  (linear projections of same input X)
  √d_k scaling: prevents softmax from entering near-zero gradient region

Multi-Head Attention:
  head■ = Attention(QW■■, KW■■, VW■■)
  MHA = Concat(head■,...,head■) · W_O
```

■ KEY IDEA

Why attention beats RNNs: (1) Parallel over sequence — no sequential bottleneck. (2) Any token can directly attend to any other in O(1) steps (RNN takes O(n) steps for distant tokens). (3) No vanishing gradient across positions. (4) Interpretable attention weights. Limitation: $O(n^2)$ memory in sequence length — for very long sequences, use FlashAttention.

4.5 Large Language Models — BERT, GPT & Fine-tuning

DL HOT

© Study Notes 2025

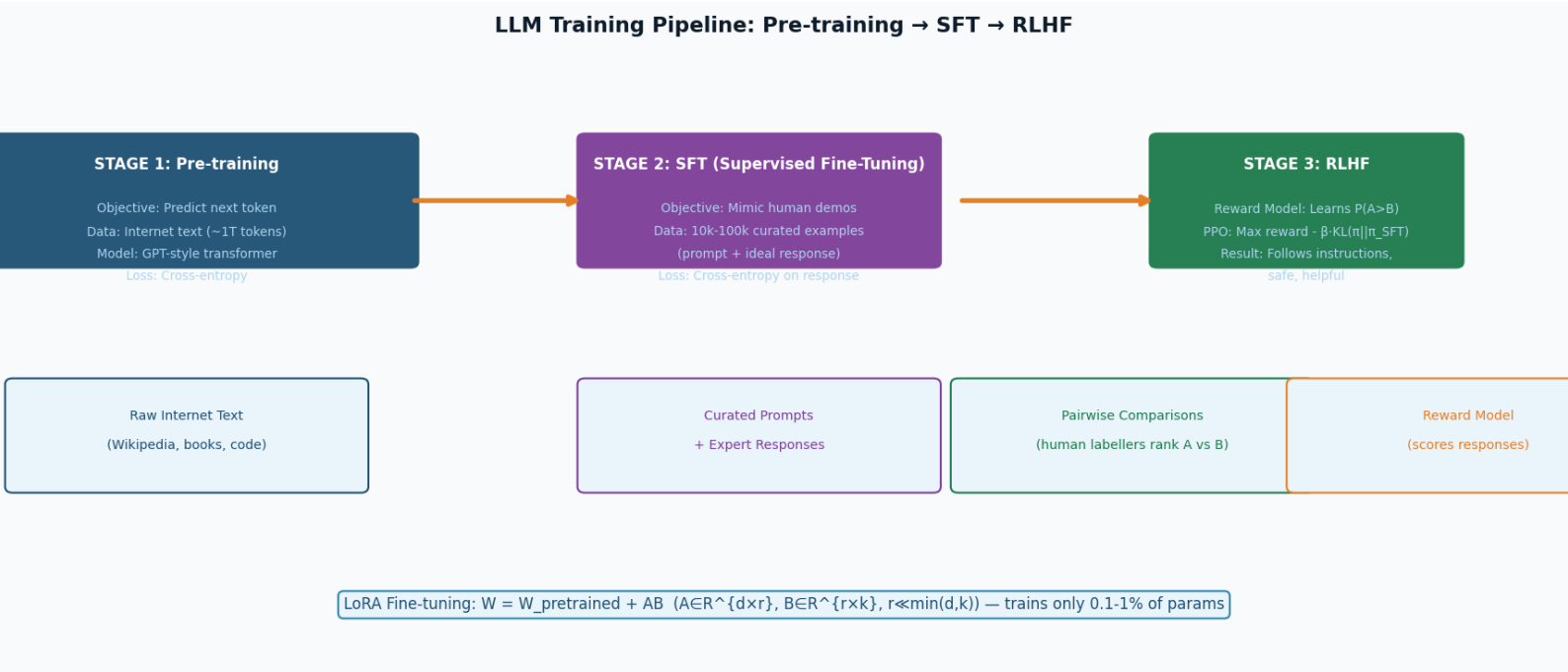

Figure 4.4: LLM training pipeline. Pre-training on internet text builds general language understanding. SFT teaches it to follow instructions. RLHF aligns it to human preferences.

Model	Architecture	Pre-training Task	Use For
BERT	Encoder-only	Masked LM (15% tokens masked) + NSP	Classification, NER, extraction, QA
GPT-3/4	Decoder-only	Causal LM (predict next token)	Generation, instruction following, code
T5	Encoder-Decoder	Span corruption denoising	Translation, summarisation, QA
LLaMA	Decoder-only	Causal LM (more data, longer)	Open-source, fine-tuning experiments

LoRA — Parameter-Efficient Fine-tuning

```
LoRA: W = W■ + AB
  W■ ∈ ■^{d×k}  (frozen pretrained weights)
  A ∈ ■^{d×r}, B ∈ ■^{r×k},  r ■ min(d,k)  (trainable)
  Init: A ~ N(0,σ), B = 0  (so ΔW=0 at start)

At inference: merge W = W■ + AB  (zero overhead!)
Typically r=4-64, reduces trainable params by 100-10000x
```

■ EXAMPLE

A GPT-3 model has 175B parameters. Full fine-tuning requires 175B × 4 bytes = 700GB GPU memory — infeasible for most. LoRA with r=8 requires training only ~25M parameters across attention layers (0.01%). QLoRA additionally quantises W■ to 4-bit, fitting a 65B model on a single 48GB GPU.

© Study Notes 2025

Chapter 5 — Natural Language Processing

Tokenisation, embeddings, sequence models, LLM prompting

5.1 Text Preprocessing & Tokenisation

DL ML

Method	Example	Notes
Word tokenisation	"Hello world" → ["Hello","world"]	OOV problem for rare words
BPE (Byte-Pair Encoding)	"tokenise" → ["token","ise"]	GPT-2/3/4, handles any word
WordPiece	"tokenise" → ["token","##ise"]	BERT — ## marks subword continuation
SentencePiece	Language-agnostic subwords	T5, LLaMA — works across languages

5.2 Word Embeddings

```
Word2Vec Skip-gram: predict surrounding words from centre word
  Objective: max Σ■ Σ_{-c≤j≤c, j≠0} log P(w■■■ | w■)
  Negative sampling: binary classification: is (word, context) a real pair?

Famous property: king − man + woman ≈ queen  (vector arithmetic!)
```

Embedding	Contextual?	Training	Key Feature
Word2Vec	No (one vector per word)	Self-supervised, fast	Semantic arithmetic, sub-billion params
GloVe	No	Global co-occurrence statistics	Global + local info combined
FastText	No (but has subwords)	Character n-grams	Handles OOV and morphology
BERT embeddings	Yes (context-aware)	MLM on large corpus	Same word = different vector in different context

5.3 RAG — Retrieval-Augmented Generation

RAG is the dominant production pattern for LLMs with private/fresh knowledge. It solves hallucination and knowledge cutoff problems.

```
RAG pipeline:
1. Index: encode documents into dense vectors using embedding model
2. Retrieve: for query q, find top-k similar docs via ANN (FAISS/ScaNN)
3. Augment: prepend retrieved docs to prompt
4. Generate: LLM generates answer grounded in retrieved context
```


© Study Notes 2025

```python
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

# Build index
model = SentenceTransformer("all-MiniLM-L6-v2")
docs = ["Amazon Q3 revenue was $143B...", "AWS grew 12% YoY..."]
embeddings = model.encode(docs, normalize_embeddings=True)

index = faiss.IndexFlatIP(embeddings.shape[1])  # inner product = cosine for normalised
index.add(embeddings.astype(np.float32))

# Query
query_emb = model.encode(["What was Amazon revenue?"], normalize_embeddings=True)
distances, indices = index.search(query_emb.astype(np.float32), k=3)
retrieved_docs = [docs[i] for i in indices[0]]
# Feed retrieved_docs + query to LLM
```


© Study Notes 2025

Chapter 6 — Statistics & Probability

Distributions, testing, Bayesian methods

6.1 Probability Foundations

STAT HOT

```
Bayes theorem: P(A|B) = P(B|A)·P(A) / P(B)
Law of total prob: P(B) = Σ■ P(B|A■)·P(A■)

Expectation: E[X] = Σx·P(x)    Variance: Var[X] = E[X²] − (E[X])²
Central Limit Theorem: X■ ~ N(μ, σ²/n)  as n→∞  (regardless of distribution!)
```

Key Distributions

Distribution	Parameters	Mean	Use For
Normal $N(\mu,\sigma^2)$	μ, σ^2	μ	Test statistics, measurement errors, many natural phenomena
Binomial(n,p)	n trials, p success prob	np	Count of successes in n trials (CTR experiments)
Poisson(λ)	λ = avg rate	λ	Count of rare events per unit time (crashes, arrivals)
Exponential(λ)	λ = rate	$1/\lambda$	Time between events (inter-arrival time)
Beta(α,β)	α, β	$\alpha/(\alpha+\beta)$	Prior/posterior for probabilities (Bayesian A/B testing)
Bernoulli(p)	p	p	Single binary outcome (click/no-click)

6.2 Hypothesis Testing

STAT HOT

```
H■: null hypothesis (no effect)   H■: alternative (effect exists)
p-value = P(data as extreme as observed | H■ true)
  NOT P(H■ is true given data)  ← most common misconception!

Type I error (α=0.05): reject H■ when it is true  → false positive
Type II error (β): fail to reject H■ when H■ true → false negative
Power = 1−β = P(detect true effect)
```

■ **NOTE**

$p < 0.05$ does NOT mean "95% probability the effect is real". It means: if there were truly no effect, this result would occur less than 5% of the time by chance. With huge sample sizes, even meaningless effects become statistically significant — always report effect size.

© Study Notes 2025

Situation	Test to Use
Compare means of 2 groups (normal, known σ)	Two-sample z-test
Compare means of 2 groups (normal, unknown σ)	Two-sample t-test
Compare means of 2 groups (non-normal or small n)	Mann-Whitney U test (non-parametric)
Compare proportions (click rates, conversion)	Two-proportion z-test
Multiple groups (>2)	ANOVA (parametric) or Kruskal-Wallis
Categorical association	Chi-squared test

6.3 Bayesian Statistics

STAT

```
Posterior ∝ Likelihood × Prior
P(θ|data) ∝ P(data|θ) · P(θ)

MLE: θ* = argmax P(data|θ)            (no prior, can overfit)
MAP: θ* = argmax P(data|θ)·P(θ)        (prior acts as regulariser!)
  L2 prior (Gaussian) ↔ Ridge regression
  L1 prior (Laplace) ↔ Lasso regression

Conjugate priors (closed-form posterior):
  Beta(α,β) prior + Binomial likelihood = Beta(α+k, β+n−k) posterior
  Gaussian prior + Gaussian likelihood = Gaussian posterior
```

■ EXAMPLE

A/B test Bayesian way: You observe 50/500 clicks (10%) in control, 60/500 (12%) in treatment. Put a Beta(1,1) prior on each conversion rate (uniform belief). Update to Beta(51,451) and Beta(61,441). Compute P(θ_B > θ_A) by sampling. This gives a direct probability statement: "There is 87% probability treatment is better."

© Study Notes 2025

Chapter 7 — A/B Testing & Causal Inference

Experiment design, power analysis, CUPED, causal methods

7.1 A/B Testing — Full Design Process

STAT HOT

A/B testing is central to Amazon and Microsoft operations. Know the full process cold.

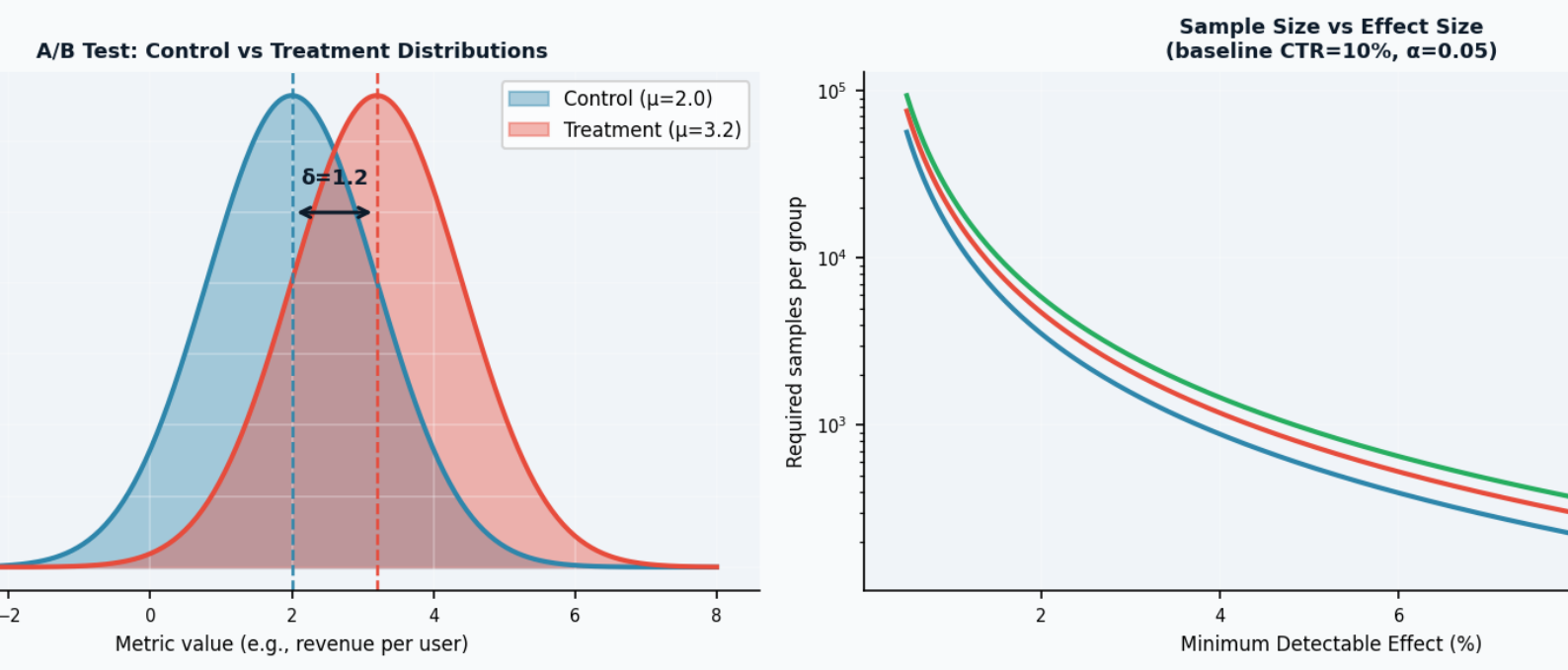

Figure 7.1: Left — overlapping distributions of control and treatment; the effect size δ determines how easy it is to detect. Right — with larger effects, far fewer samples are needed.

Step-by-Step A/B Test Design

Step	What to Do	Common Mistake
1. Hypothesis	Write H■ and H■. Define metric before running.	Changing metric after seeing results (p-hacking)
2. Sample size	Calculate $n = 2\sigma^2(z_{\alpha/2}+z_\beta)^2/\delta^2$	Running until significant = Type I inflation
3. Randomisation	Hash(user_id + experiment_id) for consistent assignment	Randomising by session/device (network effects)
4. Guardrails	Define metrics that must NOT degrade	Optimising proxy metric at expense of true goal
5. Run duration	Run ≥1 full business cycle (usually 2 weeks)	Stopping early due to novelty effect
6. Analysis	Two-sided t-test or Mann-Whitney. Check SRM first.	Ignoring sample ratio mismatch (SRM)
7. Decision	Ship if primary metric improved AND guardrails held	Ignoring practical significance for statistical

© Study Notes 2025

```
Sample size formula (two-proportion test):
n = 2 · p■(1−p■) · (z_{α/2} + z_β)² / δ²
where p■ = (p■+p■)/2, δ = p■−p■

For α=0.05, power=0.80: z_{α/2}=1.96, z_β=0.84  → (1.96+0.84)²≈7.85
Example: baseline CTR=5%, detect 0.5% lift → n≈58,000 per arm
```

CUPED — Cutting Experiment Run Time by 50%+

```
CUPED: Y_cuped = Y − θ(X_pre − E[X_pre])
  X_pre = same metric measured BEFORE experiment started
  θ* = Cov(Y, X_pre) / Var(X_pre)  (regression coefficient)
  Variance reduction: Var(Y_cuped) = Var(Y)·(1 − ρ²)  where ρ = Corr(Y, X_pre)
  If ρ=0.7 (typical): 1−0.49 = 51% variance reduction → same power with half the data!
```

■ AMAZON/MSFT

CUPED is used heavily at Microsoft, Amazon, Airbnb, Booking.com. In interviews, mention it shows you know real-world experimentation beyond textbook stats. The pre-experiment data (last 2-4 weeks) is the covariate. If a user bought a lot before, you adjust for that when estimating the treatment effect.

python

```python
import numpy as np
from scipy import stats

def cuped_test(y_ctrl, y_trt, x_pre_ctrl, x_pre_trt, alpha=0.05):
    # Fit CUPED correction on control group
    theta = np.cov(np.concatenate([y_ctrl, y_trt]),
                   np.concatenate([x_pre_ctrl, x_pre_trt]))[0,1] /             np.var(np.concatenate([x_pre_ctr

    grand_mean_pre = np.mean(np.concatenate([x_pre_ctrl, x_pre_trt]))

    # Apply CUPED
    y_ctrl_c = y_ctrl - theta * (x_pre_ctrl - grand_mean_pre)
    y_trt_c  = y_trt  - theta * (x_pre_trt  - grand_mean_pre)

    t_stat, p_val = stats.ttest_ind(y_ctrl_c, y_trt_c)
    lift = (y_trt_c.mean() - y_ctrl_c.mean()) / y_ctrl_c.mean()
    print(f"Lift: {lift:.2%}, p-value: {p_val:.4f}, Significant: {p_val < alpha}")
    return lift, p_val
```

7.2 Causal Inference

STAT HOT

When you cannot run an A/B test, you need causal inference. This is tested heavily at Amazon for policy decisions, pricing, and marketplace design.

Method	When to Use	Key Assumption	Example
RCT / A/B test	Ideal — run whenever possible	Random assignment	Test new checkout flow
Diff-in-Diff (DiD)	Treatment rolled out to some groups	Parallel trends before treatment	Policy rolled out in some states

© Study Notes 2025

Method	When to Use	Key Assumption	Example
Instrumental Variables	Unmeasured confounders exist	Instrument affects treatment but not outcome directly	Distance to hospital → healthcare usage
Propensity Score Matching	Observational data, measurable confounders	No unmeasured confounders	Treatment group vs matched controls
Regression Discontinuity	Treatment assigned by threshold	No manipulation of running variable	Scholarship effect (grade cutoff)
Synthetic Control	Single treated unit, many controls	Good pre-period fit	Estimating effect of a policy in one state

```
Difference-in-Differences:
ATT = (Treated_after − Treated_before) − (Control_after − Control_before)
  First difference removes time-invariant confounders
  Second difference removes time trends
  Key assumption: parallel trends (groups would have evolved similarly without treatment)
```

NOTE

Simpson's Paradox: A drug appears worse overall but better in every subgroup. This happens when a confounder (disease severity) is correlated with both treatment and outcome. Always check for confounders before drawing conclusions from observational data.

© Study Notes 2025

Chapter 8 — ML System Design

Design framework, real systems, MLOps

8.1 The 6-Step ML Design Framework

SYS HOT

Use this structure in every ML design interview. Interviewers want to see you think systematically.

1. Problem Formulation	What is the ML task? Classification / ranking / regression / generation? Map business metric (revenue) to ML metric (AUC, NDCG). Estimate scope (data volume, latency, update frequency).
2. Data Strategy	Sources, collection, labelling, volume, freshness. Address data quality issues. Train/val/test split strategy. Handle data leakage — especially temporal.
3. Feature Engineering	User features (history, demographics). Item features (content, metadata). Context features (time, device, location). Interaction features (user-item affinity, recency). Offline vs online features.
4. Model Selection & Training	Start simple (logistic regression baseline). Add complexity if needed. Address class imbalance. Training at scale (distributed training, sampling strategy).
5. Offline Evaluation	Offline metrics, error analysis, slice-based eval (fairness). Ablation study to justify design choices.
6. Serving & Monitoring	Latency budget (p50/p99). Batch vs real-time inference. Feature store for train-serve consistency. Monitor feature drift, prediction drift, model performance degradation.

8.2 Design: Product Recommendation System (Amazon Style)

SYS HOT

This is the most common ML design question at Amazon. Know the full pipeline.

Component	Approach	Why
Candidate Generation	Two-tower model: user embedding + item embedding, ANN retrieval (FAISS)	Reduce millions of items to 1000 candidates in <10ms
Filtering	Rule-based: already purchased, out of stock, restricted	Compliance and basic quality
Ranking	LambdaMART or DNN with user+item+context features	Order 1000 candidates optimally for this user
Re-ranking	Diversity injection, business rules, fairness constraints	Avoid showing 10 identical products
Serving	Pre-computed user embeddings refreshed hourly, real-time session features	Balance freshness and latency
Evaluation	Offline: NDCG@10, HitRate@10. Online: CTR, GMV, session depth	Offline metrics predict online uplift imperfectly

© Study Notes 2025

8.3 MLOps — Production ML Best Practices

SYS

Practice	Description
Feature Store	Central repo for features. Offline store (training) + Online store (<10ms serving). Prevents train/serve skew.
Model Registry	Track model versions, hyperparameters, metrics. Enables rollback. Examples: MLflow, W&B, SageMaker.
Shadow Deployment	Run new model alongside old model, log predictions, do NOT serve. Compare offline.
Canary Deploy	Route 1-5% traffic to new model. Monitor guardrail metrics. Gradually increase if no degradation.
Data Drift Monitoring	KS test or PSI (Population Stability Index) on feature distributions. Trigger retraining if drift detected.
Model Drift Monitoring	Monitor prediction distribution and business metrics. Separate from data drift — concept may have changed.
Retraining Pipeline	Automated: schedule-based (weekly) or trigger-based (when performance drops). Always validate before promoting.

© Study Notes 2025

Chapter 9 — Coding Patterns

Python, NumPy, PyTorch, SQL — production-ready patterns

9.1 Python / NumPy / Pandas Essentials

CODE

```python
import numpy as np
import pandas as pd

# Vectorised operations — ALWAYS prefer over loops
a = np.array([[1,2],[3,4]])
np.dot(a, a.T)           # matrix multiply: [[5,11],[11,25]]
np.linalg.eig(a)         # eigenvalues, eigenvectors
np.linalg.svd(a)         # U, S, Vt decomposition
np.einsum("ij,jk->ik", a, a)  # batched operations efficiently

# Pandas — interview staples
df.groupby("user_id").agg({"revenue": "sum", "clicks": "count"}).reset_index()
df.merge(orders, on="user_id", how="left")
df.pivot_table(values="sales", index="region", columns="category", aggfunc="mean")
pd.cut(df["age"], bins=[0,18,35,65,100], labels=["teen","young","mid","senior"])

# FAST: vectorised conditional (avoid apply + lambda)
df["label"] = np.where(df["score"] > 0.5, "positive", "negative")

# Window functions in pandas
df["cum_rev"] = df.groupby("user_id")["revenue"].cumsum()
df["rolling_7d"] = df.groupby("user_id")["revenue"].transform(
    lambda x: x.rolling(7, min_periods=1).mean()
)
```

9.2 PyTorch — Training Loop Template

© Study Notes 2025

```python
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.LayerNorm(hidden_dim),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, output_dim)
        )
    def forward(self, x):
        return self.net(x)

model = MLP(128, 256, 1).cuda()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)
loss_fn = nn.BCEWithLogitsLoss()

for epoch in range(100):
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.cuda(), yb.cuda()
        optimizer.zero_grad()
        pred = model(xb).squeeze(-1)
        loss = loss_fn(pred, yb.float())
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
    scheduler.step()

    model.eval()
    with torch.no_grad():
        val_preds = torch.sigmoid(model(X_val.cuda())).cpu().numpy()
```

9.3 SQL — Window Functions & Analytics

© Study Notes 2025

```sql
-- Window functions (tested heavily in SQL rounds)
SELECT user_id, order_date, revenue,
    ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY order_date) AS order_num,
    SUM(revenue) OVER (PARTITION BY user_id ORDER BY order_date
                       ROWS UNBOUNDED PRECEDING) AS cumulative_revenue,
    LAG(revenue, 1) OVER (PARTITION BY user_id ORDER BY order_date) AS prev_revenue,
    AVG(revenue) OVER (PARTITION BY user_id ORDER BY order_date
                       ROWS 6 PRECEDING) AS rolling_7d_avg
FROM orders;

-- Retention / Cohort analysis
WITH first_purchase AS (
    SELECT user_id, DATE_TRUNC("month", MIN(purchase_date)) AS cohort_month
    FROM purchases GROUP BY 1
),
activity AS (
    SELECT p.user_id, f.cohort_month,
           DATEDIFF("month", f.cohort_month, p.purchase_date) AS months_since_first
    FROM purchases p JOIN first_purchase f USING (user_id)
)
SELECT cohort_month, months_since_first,
       COUNT(DISTINCT user_id) AS retained_users
FROM activity GROUP BY 1, 2 ORDER BY 1, 2;

-- Top-N per group (common interview question)
WITH ranked AS (
    SELECT category, product_id, SUM(revenue) AS total_rev,
           RANK() OVER (PARTITION BY category ORDER BY SUM(revenue) DESC) AS rnk
    FROM sales GROUP BY 1, 2
)
SELECT * FROM ranked WHERE rnk <= 3;
```


© Study Notes 2025

Chapter 10 — Top 100 Interview Questions & Answer

The most frequently asked questions with detailed answers

■ TIP

How to use this section: Cover the question, attempt an answer, then read the answer. For senior roles, answers should also mention tradeoffs, real-world considerations, and how you would make decisions.

ML Theory & Fundamentals

Q1. Explain the bias-variance tradeoff. How do you detect and fix each?

Total error = Bias² + Variance + Noise. Bias is systematic error — model is too simple to capture the true pattern (underfitting). Variance is sensitivity to training data — model memorises noise (overfitting). Detect via learning curves: both train/val error high and converging = high bias; large train/val gap = high variance. Fix bias: more features, more complex model, less regularisation, add polynomial terms. Fix variance: more training data, L1/L2 regularisation, dropout, simplify model, ensemble (bagging reduces variance). Key insight: adding data always reduces variance but never reduces bias.

Q2. What is the difference between L1 and L2 regularisation? When do you use each?

L2 (Ridge) adds $\lambda\Sigma w_{■}^2$ to loss — shrinks all weights toward zero proportionally but never exactly to zero. L1 (Lasso) adds $\lambda\Sigma|w_{■}|$ — can push weights to exactly zero (sparse model, automatic feature selection). Geometric intuition: L1 ball has diamond corners on axes; the loss contour likely touches a corner first, zeroing a weight. L2 ball is smooth — solutions land near but not on axes. Use L1 when: many irrelevant features, want automatic selection. Use L2 when: all features probably relevant, features correlated (L1 arbitrarily picks one; L2 keeps both shrunk). Use Elastic Net for high-dimensional correlated features.

Q3. What is data leakage? How do you detect and prevent it?

Leakage: the model sees information during training that it would not have at prediction time. Types: (1) Target leakage — feature is computed using the target or future data. (2) Train-test leakage — preprocessing (scaling, imputation) fitted on combined train+test. (3) Temporal leakage — time series shuffled, future events predict past. Detection: suspiciously high CV score, features with correlation > 0.9 to target, performance drop between CV and holdout. Prevention: Always fit preprocessing on train set only; use temporal splits for time series; be suspicious of any feature that is unavailable at inference time.

Q4. Explain overfitting. List 5 ways to reduce it.

(1) Get more training data — most reliable fix. (2) Regularisation — L1/L2 penalise complex weights. (3) Reduce model complexity — fewer layers, features, or tree depth. (4) Dropout — randomly deactivate neurons during training to prevent co-adaptation. (5) Early stopping — stop training when validation loss starts to increase. Also: data augmentation, ensembling (bagging), cross-validation for model selection, feature selection to remove noise variables.

Q5. What is cross-validation? What type do you use for time series?

Cross-validation estimates generalisation by training on k-1 folds and validating on the remaining 1, rotating k times, and averaging. k=5 or 10 is standard. For time series: NEVER shuffle — use TimeSeriesSplit (sklearn).

© Study Notes 2025

Each fold trains on past data and validates on the immediately following period. This respects temporal ordering and prevents future data from leaking into training. Stratified k-fold is used for imbalanced classification to maintain class ratio in every fold.

Q6. How do you handle an extremely imbalanced dataset (99% negative, 1% positive)?

Metric: use AUC-PR or F1 — accuracy is meaningless (99% accurate by predicting all negative). Data level: SMOTE (synthetic oversampling via interpolation in feature space), random undersampling, Tomek links (remove majority samples near boundary). Algorithm level: class_weight="balanced" in sklearn (inversely proportional to frequency), threshold tuning (move from 0.5 based on FP/FN cost), focal loss (down-weights easy negatives). Most practical: class_weight + AUC-PR evaluation is the first thing to try.

Q7. Explain SHAP values. Why are they better than permutation importance?

SHAP (Shapley Additive exPlanations): assigns each feature a contribution to the prediction using game theory. For each prediction, ϕ_i = average marginal contribution of feature i across all possible subsets of other features. Properties: efficiency (contributions sum to total prediction minus baseline), consistency (more important feature gets higher SHAP), local accuracy (explains individual predictions). Better than permutation importance because: (1) per-prediction explanation, not just global. (2) Handles correlated features correctly. (3) Shows direction (positive or negative impact). (4) TreeSHAP is O(n) exact, not approximation.

Q8. What is the difference between parametric and non-parametric models?

Parametric: fixed number of parameters regardless of data size. Model form is specified. Examples: linear regression (d+1 params), logistic regression, Gaussian. Pros: fast inference, interpretable, regularisable, works with small data. Non-parametric: complexity grows with data. No assumed form. Examples: KNN, Gaussian processes, kernel SVM. Pros: more flexible, fewer assumptions. Cons: slow inference for large n, harder to interpret, require more data. Deep neural networks are parametric but so high-dimensional they behave like non-parametric models.

Q9. Explain gradient descent. What is the difference between batch, stochastic, and mini-batch?

Gradient descent: update $w \leftarrow w - \eta \cdot \nabla L(w)$ iteratively. Batch GD: compute gradient on ALL data per step — exact gradient, stable, but slow and cannot fit large datasets. Stochastic GD (SGD): one sample per step — fast, noisy gradient, can escape local minima, enables online learning. Mini-batch: 32-512 samples per step — best of both: parallelisable on GPU, less noisy than SGD, faster than batch. In practice, mini-batch with Adam is the default for deep learning. The noise in SGD/mini-batch acts as implicit regularisation and helps escape sharp minima.

Q10. What is the kernel trick and why is it powerful?

The kernel trick allows SVMs (and other algorithms) to operate in very high-dimensional (even infinite-dimensional) feature spaces without explicitly computing the feature mapping $\phi(x)$. Any algorithm that only uses inner products $x_i \cdot x_j$ can be "kernelised" by replacing with $K(x_i,x_j) = \phi(x_i) \cdot \phi(x_j)$. The RBF kernel $\exp(-\gamma||x-z||^2)$ corresponds to an infinite-dimensional feature space — a linear classifier in that space = non-linear classifier in original space. This is powerful because we get the expressive power of infinite dimensions at the cost of computing $K(x_i,x_j)$ for n^2 pairs.

Deep Learning

© Study Notes 2025

Q1 1. **Explain backpropagation. How does it work and why can gradients vanish?**

Backprop is the chain rule applied to a computational graph. Forward pass: compute predictions and loss. Backward pass: starting from the loss, compute ∂L/∂w for every parameter using the chain rule. For layer l: ∂L/∂w■ = ∂L/∂a■ · ∂a■/∂z■ · ∂z■/∂w■. The error signal δ = ∂L/∂z is propagated backward: δ■ = W■■■■δ■■■ · σ'(z■). Vanishing gradients: sigmoid and tanh have derivatives ≤ 0.25. Multiplying many such small numbers through many layers → gradient ≈ 0. Fix: ReLU (gradient=1 for positive inputs), residual connections (+1 in gradient), batch normalisation, proper initialisation (He or Xavier).

Q1 2. **What is the transformer architecture? Explain attention in detail.**

Transformer is a stack of identical blocks, each containing: (1) Multi-head self-attention. (2) Position-wise feed-forward network. (3) Residual connections and layer norm after each. Attention: compute Q=XW_Q, K=XW_K, V=XW_V. Score = softmax(QK■/√d_k)·V. The √d_k scaling prevents softmax saturation. Multi-head: run h attention heads in parallel with different projections, concatenate outputs. Each head can attend to different types of relationships. Why it works: any token can directly attend to any other (O(1) path length), fully parallelisable, attention weights are interpretable.

Q1 3. **What is the difference between BERT and GPT? Which would you use for what task?**

BERT: encoder-only, bidirectional (sees left AND right context). Pre-trained with Masked LM (15% tokens masked, predict them) and NSP. Best for: classification (spam, sentiment), NER, question answering (extractive), semantic similarity. GPT: decoder-only, unidirectional (causal — only sees left context). Pre-trained with next-token prediction. Best for: text generation, summarisation, instruction following, code generation. Rule of thumb: if you need to UNDERSTAND text → BERT. If you need to GENERATE text → GPT. Modern trend: GPT-style decoder-only models (LLaMA, GPT-4) with instruction fine-tuning cover most tasks.

Q1 4. **Explain LoRA. Why is it better than full fine-tuning for LLMs?**

LoRA (Low-Rank Adaptation) keeps the pretrained weights W■ frozen and adds a low-rank decomposition ΔW = AB where A∈■^{d×r}, B∈■^{r×k}, r ■ min(d,k). Only A and B are trained. Benefits: (1) 100-10000x fewer trainable parameters (e.g., GPT-3: 175B → ~25M). (2) No additional inference latency — merge: W = W■ + AB at deploy time. (3) Preserves general capabilities of pretrained model (no catastrophic forgetting). (4) Can swap multiple LoRA adapters for different tasks without reloading base model. QLoRA combines with 4-bit quantisation of W■ — fine-tune 65B models on a single GPU.

Q1 5. **What causes the vanishing/exploding gradient problem and how is it solved?**

Vanishing: in deep networks, gradients are products of many Jacobians. If each factor < 1 (sigmoid: max derivative = 0.25), gradients → 0 exponentially with depth. Exploding: if factors > 1 (e.g., recurrent weights), gradients → ∞. Solutions for vanishing: (1) ReLU — derivative is 1 for positive inputs, not < 1. (2) Residual connections — ∂L/∂x contains +1 term from skip. (3) Batch/layer normalisation — keeps activations in a range where gradients are well-scaled. (4) LSTM gating — cell state uses addition, not multiplication. Solutions for exploding: gradient clipping (if ||g|| > threshold, scale g down to threshold).

Q1 6. **What is batch normalisation? Where do you use it and where not?**

Batch Norm normalises each mini-batch: x■ = (x–μ_B)/√(σ²_B+ε), then applies learnable scale/shift: y = γx■+β. Benefits: (1) Allows higher learning rates — less sensitive to initialisation. (2) Slight regularisation effect —

© Study Notes 2025

mini-batch statistics add noise. (3) Reduces internal covariate shift (debated). Use in: CNNs (after conv, before activation), MLPs. Do NOT use in: Transformers — use Layer Norm instead. Why? In transformers, batch size may be 1 at inference, and different positions in a sequence should be independently normalised. Layer Norm normalises over feature dimension, not batch.

Q17. Explain attention masking in transformers.

Two types of masks: (1) Padding mask — in batched training, sequences are padded to same length. Padding positions should not be attended to (they carry no real information). Set logits at padding positions to $-\infty$ before softmax → attention weights → 0. (2) Causal/autoregressive mask (used in GPT) — position t should NOT attend to positions t+1,...,n (future). Implemented as an upper-triangular matrix of $-\infty$. This makes the model autoregressive — each token can only see its left context, enabling valid next-token prediction during training and generation.

Q18. What is knowledge distillation and when is it used?

Transfer knowledge from a large "teacher" model to a smaller "student" model. Instead of training the student on hard labels (one-hot), train it to match the teacher's softmax output probabilities. The soft probabilities carry more information: e.g., 0.8 dog, 0.15 wolf, 0.05 cat tells the model that dogs and wolves are similar. Loss = $\alpha \cdot$CE(student, hard_labels) + $(1-\alpha) \cdot$KL(student_soft||teacher_soft). Temperature T scales logits to soften probabilities. Used for: model compression for edge deployment, DistilBERT (40% smaller, 97% BERT quality), distilling ensemble predictions.

Q19. How do diffusion models work?

Diffusion models have two processes: (1) Forward: gradually add Gaussian noise to data over T steps. q(x■|x■) = N(x■; √■■x■, (1–■■)I). After T steps, x■ ≈ N(0,I) (pure noise). (2) Reverse: train a U-Net to predict and remove the noise at each step. Training: sample x■ by adding noise to real data, train ε_θ(x■,t) to predict the added noise ε. Loss = $||\varepsilon - \varepsilon_\theta(x■,t)||^2$. Sampling: start from noise, iteratively denoise using learned model. DDIM enables deterministic, faster sampling (~20 steps vs 1000). Diffusion models beat GANs on image quality/diversity with more stable training.

Q20. Explain dropout. What happens at training vs inference?

Training: randomly set each neuron to 0 with probability p (dropout rate). The remaining neurons are scaled by 1/(1–p) to keep expected output constant (inverted dropout). This forces the network to learn redundant representations and prevents co-adaptation. Effect: acts like training an ensemble of 2^n different networks. Inference: all neurons are active. With inverted dropout, no scaling is needed at inference time. Typical rates: 0.1-0.3 for transformers, 0.5 for fully-connected layers. DropPath (stochastic depth): drops entire residual blocks — used in vision transformers and modern architectures.

Statistics & A/B Testing

Q21. What is a p-value? What is the most common misconception?

p-value = P(observing data at least as extreme as we got, given H■ is true). Most common misconception: p-value is NOT P(H■ is true). It is NOT the probability the result happened by chance. It does NOT measure effect size.

© Study Notes 2025

Correct: "If there were truly no difference, we would see results this extreme only 5% of the time by chance." Important: with large sample sizes, tiny meaningless effects become statistically significant. Always report effect size (Cohen's d, relative lift) AND confidence interval alongside the p-value.

Q22. Explain Type I and Type II errors with a real-world example.

Type I (false positive, rate = α): reject H_0 when H_0 is true. Example: conclude a drug works when it does not → patients receive an ineffective (possibly harmful) drug. In A/B testing: declare treatment better when it actually is not → ship a bad feature. Type II (false negative, rate = β): fail to reject H_0 when H_0 is true. Example: miss a drug that actually works → patients are denied effective treatment. In A/B testing: miss a winning feature → lost revenue opportunity. Power = $1-\beta$. The tradeoff: lowering α (fewer false positives) increases β (more false negatives). Choose based on which error is more costly.

Q23. How do you calculate sample size for an A/B test?

Formula: $n = 2\cdot\sigma^2\cdot(z_{\alpha/2}+z_\beta)^2/\delta^2$ per arm. For proportions: $\sigma^2 \approx p_0(1-p_0)$. For $\alpha=0.05$ (two-sided), power=0.80: $(1.96+0.84)^2\approx7.85$. Example: baseline CTR=10%, want to detect 1% absolute lift. $n = 2\cdot0.105\cdot0.895\cdot7.85/(0.01)^2 \approx$ 147,000 per arm. Practical tips: (1) MDE (minimum detectable effect) is the most impactful parameter — halving MDE quadruples sample size. (2) Higher power requires more data. (3) One-sided tests require less data but are only valid if you would never ship if the result goes the wrong direction. (4) Use simulation or an online calculator (Evan Miller) to double-check.

Q24. What is CUPED and how does it work?

CUPED (Controlled-experiment Using Pre-Experiment Data) is a variance reduction technique. Use pre-experiment observations X of the same metric to reduce variance of the outcome Y. $Y_{cuped} = Y - \theta(X - E[X])$ where $\theta = Cov(Y,X)/Var(X)$. This has the same expectation as Y (unbiased) but lower variance: $Var(Y_{cuped}) = Var(Y)\cdot(1-\rho^2)$. If the pre-experiment correlation $\rho=0.7$, variance is reduced by 51% — equivalent to having 2x the data, or cutting the experiment duration in half. Used extensively at Microsoft, Amazon, Booking.com, Airbnb. Implementation: compute θ on the pooled data, subtract the correction, then run standard t-test on Y_{cuped}.

Q25. What is the difference-in-differences method? What is its key assumption?

DiD estimates causal effects in observational settings where treatment is assigned to some groups. ATT = (Treated_after – Treated_before) – (Control_after – Control_before). First difference removes time-invariant confounders within each group. Second difference removes time trends common to both groups. Key assumption: parallel trends — in the absence of treatment, both groups would have evolved in the same way. Check by plotting pre-treatment trends: they should be parallel. Violation example: if treated group was already trending upward before treatment, DiD will overestimate the effect. Event-study plots are used to test this assumption.

Q26. Explain Bayesian A/B testing vs frequentist. When would you use each?

Frequentist: pre-specify α and sample size, run test, get p-value, binary reject/fail-to-reject decision. Cannot look at results before planned end. Cannot make probability statements about which variant is better. Bayesian: compute P(B is better than A | data). Can monitor continuously (sequential testing). Gives intuitive probability statements: "87% probability treatment lifts revenue." Can incorporate prior knowledge (e.g., past experiments suggest 2-3% lifts are typical). Use Bayesian when: product teams want interpretable probabilities; want to stop early; want to incorporate prior information. Use frequentist when: strict error control needed (medical, regulated domains); organisation already standardised on frequentist framework.

© Study Notes 2025

Q27. What is Simpson's Paradox?

A trend appears in every subgroup but reverses or disappears in the combined data. Example: Hospital A has 30% mortality rate, Hospital B has 20%. But A is actually BETTER for both mild and severe cases — Hospital A just receives more severe cases (confounding variable). The overall rate is a weighted average, and B has more mild cases. Resolution: always stratify by confounders when comparing groups. In ML: a model may appear to perform well overall but perform worse than baseline for minority subgroups. Always do slice-based evaluation. The lesson: aggregate statistics can be deeply misleading when the data comes from heterogeneous subpopulations.

Q28. What is propensity score matching?

Propensity score = P(treatment=1 | X) — the probability of being in the treated group given covariates. Computed with logistic regression on observational data. Matching: find each treated unit a control unit with a similar propensity score. After matching, the treated and control groups should be balanced on observed covariates — as if they were randomly assigned. Then estimate ATT = mean(Y_treated) – mean(Y_matched_control). Key assumption: no unmeasured confounders (ignorability given X). Limitation: cannot control for variables you did not measure. Always check covariate balance (standardised mean differences) after matching.

ML System Design

Q29. Design a fraud detection system for a payment platform.

Two-tier system: (1) Real-time (<100ms): rule-based + lightweight model (LightGBM on velocity/amount/device features) blocks obvious fraud. (2) Batch: graph neural network or LSTM on transaction sequences, reviewed by analysts. Features: transaction amount, merchant category, location (is it unusual for this user?), velocity (5 transactions in 1 minute), device fingerprint, IP reputation, time of day. Labels: chargeback signals (come 30-90 days later — delayed labels). Challenge: 0.1% fraud rate — use class_weight, AUC-PR metric. Feedback loop: analyst decisions become new training labels. Deploy with shadow mode first, monitor FPR carefully (blocking legitimate transactions is very damaging).

Q30. How would you build a search ranking system for Amazon?

Multi-stage pipeline: (1) Query understanding: spell correction, entity recognition, intent classification (navigational vs informational), query expansion with synonyms. (2) Retrieval: BM25 inverted index + dense retrieval (dual encoder). Combine scores. (3) Ranking: LambdaMART or DNN. Features: query-document TF-IDF/BM25, semantic similarity (BERT), document quality (reviews, sales velocity, Prime eligibility), user behaviour (historical CTR for this query-product pair), personalisation signals. (4) Re-ranking: diversity, sponsored results, freshness. Evaluation: offline NDCG@10 with human judgements, online interleaving experiments (low noise), full A/B test.

Q31. Explain the two-tower architecture for recommendation.

Two-tower: user tower and item tower independently encode their respective inputs into a shared embedding space. User tower: takes user ID, history of clicked/purchased items, demographics, device. Item tower: takes item ID, title, category, price, ratings. Score = cosine_similarity(user_embedding, item_embedding) or dot product. Training: in-batch negatives (all items in batch except the one the user interacted with are treated as negatives) or sampled negatives. Serving: pre-compute all item embeddings offline. For a given user query, find top-k items using approximate nearest neighbour (FAISS IndexFlatIP). Benefits: user and item embeddings can be computed

© Study Notes 2025

independently and cached. New items can be added without retraining user side.

Q32. How do you handle the cold start problem in recommendations?

Cold start = new users or new items with no interaction history. New user: (1) Ask for explicit preferences during onboarding. (2) Use demographic features (age, location) for content-based initial recommendations. (3) Default to popular/trending items. (4) Use contextual bandits — explore items to learn preferences quickly while exploiting known preferences. New item: (1) Content-based features (description, category, price) for initial retrieval. (2) Manually curate metadata to ensure discoverability. (3) Expose to a small random sample of users immediately to gather initial signals. (4) Once a few interactions exist, collaborative filtering kicks in.

Q33. Design a content moderation system at scale.

Multi-modal, multi-stage system: (1) Rule-based (fastest, highest precision): keyword matching, known bad URLs, hash-based detection of known CSAM. (2) ML classifier (text: BERT; image: ViT/CLIP): recall-focused, flag everything above threshold for review. (3) Human review queue: model uncertainty as priority signal — review borderline cases first. (4) Appeals pipeline for false positives. Training: multilabel classification (hate, spam, misinformation, NSFW). Challenges: adversarial users (adapt constantly), class imbalance, need for low latency (<200ms for comments), multilingual content. Evaluation: precision and recall per category, false positive rate (must be very low for reputable creators), bias audits across demographics.

Q34. What is a feature store and why is it important?

A feature store is a centralised system for storing, serving, and managing ML features. Components: (1) Offline store (data warehouse: Hive, BigQuery) — historical data for model training, supports point-in-time queries to avoid leakage. (2) Online store (Redis, DynamoDB) — low-latency (<10ms) serving for real-time inference. (3) Feature registry — metadata, documentation, ownership, monitoring. (4) Ingestion pipelines — batch + streaming. Why important: (1) Prevents training-serving skew (same feature computation in both). (2) Feature reuse across teams (compute once, use many). (3) Point-in-time correctness ensures training data does not leak future information. (4) Monitoring built in. Examples: Feast (open source), Tecton, SageMaker Feature Store.

Q35. How do you detect and handle model drift in production?

Two types: (1) Data drift (covariate shift): input feature distribution P(X) changes. Example: user demographics shift, new device types appear. Detect: compare feature distributions between training and serving data using KS test or PSI (Population Stability Index). PSI > 0.2 = significant drift. (2) Concept drift: relationship P(Y|X) changes. Example: user preferences changed due to seasonality, world event, product catalogue change. Detect: monitor model performance metrics (AUC, NDCG) — if labels available. If labels delayed: monitor prediction distribution shift. Response: (1) Scheduled retraining (weekly/monthly). (2) Triggered retraining when drift exceeds threshold. (3) Warm-start retraining from current model, not from scratch.

Coding & Algorithms

Q36. Implement k-means clustering from scratch.

© Study Notes 2025

```python
def kmeans(X, k, n_iters=100, tol=1e-4, seed=42):
    np.random.seed(seed)
    # K-Means++ initialisation (better than random)
    centroids = [X[np.random.randint(len(X))]]
    for _ in range(k - 1):
        dists = np.min([np.sum((X - c)**2, axis=1) for c in centroids], axis=0)
        probs = dists / dists.sum()
        centroids.append(X[np.random.choice(len(X), p=probs)])
    centroids = np.array(centroids)

    for iteration in range(n_iters):
        # E-step: assign points to nearest centroid
        dists = np.linalg.norm(X[:, None] - centroids[None, :], axis=2)
        labels = np.argmin(dists, axis=1)
        # M-step: update centroids
        new_c = np.array([X[labels == j].mean(axis=0) for j in range(k)])
        if np.max(np.abs(new_c - centroids)) < tol:
            break
        centroids = new_c
    return labels, centroids
```

Q3 7. **Implement softmax and cross-entropy loss from scratch.**

```python
def softmax(x):
    # Subtract max for numerical stability (prevents overflow)
    e = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)

def cross_entropy_loss(logits, y_true):
    # y_true: integer class labels
    n = len(y_true)
    probs = softmax(logits)
    # Only sum probabilities at true class (log likelihood)
    log_probs = -np.log(probs[np.arange(n), y_true] + 1e-9)
    return log_probs.mean()

def cross_entropy_grad(logits, y_true):
    probs = softmax(logits)
    probs[np.arange(len(y_true)), y_true] -= 1
    return probs / len(y_true)  # gradient w.r.t. logits
```

Q3 8. **Write SQL to calculate 30-day rolling retention.**

© Study Notes 2025

```sql
-- For each user, find their first activity date (cohort)
WITH first_active AS (
    SELECT user_id, MIN(activity_date) AS cohort_date
    FROM user_events GROUP BY user_id
),
-- Tag each activity with days since first active
tagged AS (
    SELECT e.user_id, f.cohort_date,
           DATEDIFF(e.activity_date, f.cohort_date) AS day_offset
    FROM user_events e
    JOIN first_active f USING (user_id)
)
-- Count users retained at each day offset
SELECT
    cohort_date,
    day_offset,
    COUNT(DISTINCT user_id) AS retained,
    COUNT(DISTINCT user_id) /
        FIRST_VALUE(COUNT(DISTINCT user_id)) OVER
        (PARTITION BY cohort_date ORDER BY day_offset) AS retention_rate
FROM tagged
WHERE day_offset IN (0, 1, 7, 14, 30)
GROUP BY 1, 2;
```

Q39. **Implement logistic regression with gradient descent from scratch.**

```python
class LogisticRegression:
    def __init__(self, lr=0.01, n_iters=1000, l2=0.0):
        self.lr, self.n_iters, self.l2 = lr, n_iters, l2

    def _sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

    def fit(self, X, y):
        n, d = X.shape
        self.w = np.zeros(d); self.b = 0.0
        for _ in range(self.n_iters):
            z = X @ self.w + self.b
            y_hat = self._sigmoid(z)
            err = y_hat - y
            self.w -= self.lr * (X.T @ err / n + self.l2 * self.w)
            self.b -= self.lr * err.mean()

    def predict_proba(self, X):
        return self._sigmoid(X @ self.w + self.b)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)
```

Q40. **How would you efficiently find duplicate records in a large dataset?**

© Study Notes 2025

```python
import pandas as pd
import hashlib

# Method 1: Exact duplicates using pandas
duplicates = df[df.duplicated(subset=["col1","col2","col3"], keep=False)]

# Method 2: Fuzzy matching at scale using MinHash LSH
# For large datasets where exact match fails (typos, formatting)
from datasketch import MinHash, MinHashLSH

def get_minhash(text, num_perm=128):
    m = MinHash(num_perm=num_perm)
    for word in text.lower().split():
        m.update(word.encode("utf8"))
    return m

lsh = MinHashLSH(threshold=0.8, num_perm=128)
for idx, row in df.iterrows():
    lsh.insert(idx, get_minhash(row["text"]))

# Find near-duplicates for a query
query = get_minhash("some customer name with typo")
similar_ids = lsh.query(query)  # O(1) approximate lookup
```

Generative Models & Advanced Topics

Q4 1. **Explain VAEs vs GANs vs Diffusion Models. When would you use each?**

VAE: encoder maps input to a latent distribution $q(z|x) = N(\mu,\sigma^2)$, decoder reconstructs from sampled z. Loss = reconstruction loss + $KL(q(z|x)||N(0,I))$. Reparameterisation trick enables backprop. Pros: stable training, explicit latent space, interpolation. Cons: blurry outputs. GAN: generator G(z) tries to fool discriminator D(x). Adversarial training is unstable (mode collapse, training instability). Pros: sharp images. Cons: training can fail, no explicit likelihood. Diffusion: add noise iteratively then learn to denoise. Pros: stable training, highest quality/diversity, invertible. Cons: slow sampling (use DDIM for 10-50 steps). Use: VAE for latent space exploration, GAN for fast high-quality samples (legacy), Diffusion for best quality image/audio generation.

Q4 2. **What are mixture of experts (MoE) models and how do they scale?**

MoE replaces the dense FFN in each transformer block with N expert FFNs. A learned router selects the top-k experts per token (k=1 or 2 typically). Only those experts process the token. Benefits: scale parameters without proportionally scaling compute. Mixtral 8×7B has 46.7B total parameters but only activates ~13B per token — similar compute to a 13B dense model but quality of a 40B+ model. Training challenges: load balancing (auxiliary loss prevents all tokens routing to one expert), communication overhead in distributed settings. The router learns that different experts specialise in different types of content.

Q4 3. **Explain RLHF and why it is needed for LLMs.**

RLHF (Reinforcement Learning from Human Feedback) aligns LLM behaviour with human preferences. Three stages: (1) SFT: fine-tune pretrained LLM on high-quality demonstrations (prompts + ideal responses). (2) Reward Model: collect human pairwise preferences (response A vs B). Train RM with Bradley-Terry: $P(A \text{ preferred}) = \sigma(r(A)-r(B))$. (3) PPO: optimise LLM to maximise expected reward while staying close to SFT model: $\max E[r(x,y)] - \beta \cdot KL(\pi||\pi_SFT)$. The KL penalty prevents reward hacking (LLM finds adversarial responses that maximise RM

© Study Notes 2025

score but are undesirable). DPO is a simpler alternative that directly optimises on preference data without explicit RM training.

Q4 4. **What is speculative decoding and how does it speed up LLM inference?**

Problem: LLM autoregressive generation is sequential — each token requires a full forward pass through the large model. Speculative decoding: use a small "draft" model (e.g., 7B) to generate K candidate tokens quickly, then pass all K tokens to the large "target" model (e.g., 70B) for verification in a single forward pass (parallelisable). Accept tokens where the target model agrees; reject and resample from the first disagreement. Key property: the output distribution is exactly the same as if the target model generated each token sequentially. Speed: if most tokens are accepted, you get K tokens per large model call instead of 1. In practice, 3-5x speedup with a good draft model.

© Study Notes 2025

8-Week Interview Preparation Roadmap

Structured daily study plan for Amazon & Microsoft

Week 1-2 ML Foundations + Algorithms

- Day 1-2: Bias-variance, regularisation, feature engineering (Ch 1)
- Day 3-4: Evaluation metrics — implement precision/recall/AUC from scratch
- Day 5-6: Gradient descent variants — implement SGD, Adam from scratch
- Day 7-8: Linear/logistic regression + SVM — derive gradients
- Day 9-10: Decision trees, Random Forest, XGBoost — train on Kaggle dataset
- Day 11-14: Model comparison + 5 LeetCode mediums daily

Week 3-4 Deep Learning + Transformers

- Day 15-16: Backpropagation — implement 2-layer net from scratch in NumPy
- Day 17-18: CNNs — implement conv layer, build ResNet in PyTorch
- Day 19-20: RNNs, LSTMs — implement LSTM cell from scratch
- Day 21-22: Transformers — implement scaled dot-product attention from scratch
- Day 23-24: BERT and GPT — read original papers, implement tokenisation
- Day 25-28: LLMs, LoRA, RLHF — read InstructGPT, LoRA papers

Week 5-6 Statistics + A/B Testing + Causal Inference

- Day 29-30: Probability distributions — solve 20 problems from scratch
- Day 31-32: Hypothesis testing — implement two-sample t-test, power analysis
- Day 33-34: Bayesian statistics — implement Beta-Binomial conjugate update
- Day 35-36: A/B testing design — design a full experiment end-to-end
- Day 37-38: CUPED — implement and show variance reduction on simulated data
- Day 39-42: Causal inference — DiD, IV, RD — work through case studies

Week 7-8 System Design + Coding + Mock Interviews

- Day 43-44: ML system design framework — practice with rec system
- Day 45-46: Design fraud detection + search ranking from scratch
- Day 47-48: MLOps — feature stores, drift detection, deployment
- Day 49-50: SQL window functions — solve 30 HackerRank SQL problems
- Day 51-52: Python/pandas — implement ML algorithms from scratch
- Day 53-56: 4 full mock interviews — record and review each one

Essential Resources

Resource	Best For	Priority
ESLII + PRML (Bishop)	Deep ML theory	***
Dive into Deep Learning (d2l.ai)	DL with runnable code	***
CS229 Stanford lecture notes	ML math foundations	***

© Study Notes 2025

Resource	Best For	Priority
Designing ML Systems (Chip Huyen)	ML system design	***
"Trustworthy Online Controlled Experiments"	A/B testing at scale	***
Attention is All You Need (Vaswani 2017)	Original transformer paper	***
InstructGPT / RLHF paper (OpenAI 2022)	LLM alignment	**
LoRA paper (Hu et al. 2021)	Efficient fine-tuning	**
LeetCode Premium (focus: arrays, DP, graphs)	Coding rounds	***
Amazon Leadership Principles	Behavioural rounds	***

© Study Notes 2025